

项目过程(深度学习)

配置环境

配置计算环境

驱动、CUDA、cuDNN、框架四者版本必须严格匹配

原因：

NVIDIA 驱动版本需支持目标 CUDA 版本（CUDA 12.x 需要驱动版本 $\geq$ 525.60）。

cuDNN 必须与 CUDA 版本对应。

深度学习框架（如 PyTorch/TensorFlow）可能仅支持特定 CUDA 版本（如 TensorFlow 2.15 仅支持 CUDA 12.0）。

操作：

访问框架官网查看支持的 CUDA 版本（如 [TensorFlow 版本表](#)）。

通过 [NVIDIA 文档](#) 确认 CUDA 与 cuDNN 的兼容性。

CUDA 安装路径不要包含中文或空格

原因：

默认路径为：C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v11.8 (11.8 为 CUDA 的版本)

自定义路径可能导致[环境变量](#)配置错误

我的

```
C:\Users\qixua>nvidia-smi
Mon Mar  9 11:22:35 2026

+-----+
| NVIDIA-SMI 531.88                 Driver Version: 531.88          CUDA Version: 12.1         |
+-----+-----+
| GPU   Name                               TCC/WDDM | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf              Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
|  0  NVIDIA GeForce RTX 2050           WDDM  | 00000000:01:00.0 Off |          N/A         |
| N/A   40C   P8              4W /  N/A      | 19MiB /  4096MiB |      0%    Default  |
|=====+=====+
|
+-----+
| Processes:                                     GPU Memory |
|  GPU   GI    CI          PID    Type    Process name                  Usage       |
|=====+=====+
|
```

cuDNN Package [1]	CUDA Toolkit Version	Supports static linking? [2]	NVIDIA Driver Version for Linux	NVIDIA Driver Version for Windows	CUDA Compute Capability	Supported NVIDIA GPU Architectures
cuDNN 9.19.1 for CUDA 13.x	13.1 13.0	Yes	>=580.65.06	N/A	12.0 10.0 9.0 8.9 8.6 8.0 7.5	- NVIDIA Blackwell - NVIDIA Hopper - NVIDIA Ada Lovelace - NVIDIA Ampere - NVIDIA Turing
cuDNN 9.19.1 for CUDA 12.x	12.9 12.8 12.6 12.5 12.4 12.3 12.2 12.1 12.0	Yes	>=525.60.13	>=527.41	12.0 [3] 10.0 [3] 9.0 8.9 8.6 8.0 7.5	- NVIDIA Blackwell [3] - NVIDIA Hopper - NVIDIA Ada Lovelace - NVIDIA Ampere - NVIDIA Turing

发现支持最高 12.1 版本的 CUDA，搜支持的 PyTorch 版本：

PyTorch 2.5.0: CUDA 11.8、12.1、12.4

PyTorch 2.4.x: CUDA 11.8、12.1、12.4

PyTorch 2.3.x: CUDA 11.8、12.1

确定 Python 版本：

PyTorch 2.5.1 支持以下 Python 版本：

Python 3.9

Python 3.10

Python 3.11

Python 3.12

Python 3.13

确定 cuDNN 版本 9.19.1

安装

[CUDA Toolkit Archive](#) | [NVIDIA Developer](#)

[CUDA 工具包文档 12.1](#)

CUDA Toolkit 12.1 Update 1 Downloads

Select Target Platform

Click on the green buttons that describe your target platform. Only supported platforms will be shown. By downloading and using the software, you agree to fully comply with the terms and conditions of the [CUDA EULA](#).

Operating System	Linux Windows
Architecture	x86_64
Version	10 11 Server 2019 Server 2022
Installer Type	exe (local) exe (network)

Download Installer for Windows 11 x86_64

The base installer is available for download below.

> Base Installer

Download (3.2 GB)

Installation Instructions:

1. Double click cuda_12.1.1_531.14_windows.exe

2. Follow on-screen prompts

The checksums for the installer and patches can be found in [Installer Checksums](#).
For further information, see the [Installation Guide for Microsoft Windows](#) and the [CUDA Quick Start Guide](#).



"Visual Studio Integration"用于将 CUDA 工具链与 Visual Studio 开发环境集成的组件，如果没有安装 Visual Studio，不需要安装此组件以避免安装 CUDA 失败



CUDA 安装包自带的显卡驱动版本（如下图 551.78）可能与 当前系统安装的显卡驱动版本（如下图 555.85）存在差异，覆盖安装可能会导致蓝屏、卡顿或兼容性问题；HD Audio 是 NVIDIA 显卡的 HDMI/DP 接口音频输出驱动，与 CUDA 的 GPU 计算无关，所以不需要安装

重启，之后检查，如果没有可能是环境变量的问题


```
C:\Users\qixua>nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005–2023 NVIDIA Corporation
Built on Mon_Apr__3_17:36:15_Pacific_Daylight_Time_2023
Cuda compilation tools, release 12.1, V12.1.105
Build cuda_12.1.r12.1/compiler.32688072_0
```

[cuDNN Archive | NVIDIA Developer](#)

Download cuDNN v8.9.7 (December 5th, 2023), for CUDA 12.x

Local Installers for Windows and Linux, Ubuntu(x86_64, armsbsa)

[Local Installer for Windows \(Zip\)](#)

[Local Installer for Linux x86_64 \(Tar\)](#)

[Local Installer for Linux PPC \(Tar\)](#)

[Local Installer for Linux SBSA \(Tar\)](#)

[Local Installer for Debian 11 \(Deb\)](#)

[Local Installer for Ubuntu20.04 x86_64 \(Deb\)](#)

[Local Installer for Ubuntu22.04 x86_64 \(Deb\)](#)

[Local Installer for Ubuntu20.04 aarch64sbsa \(Deb\)](#)

[Local Installer for Ubuntu22.04 aarch64sbsa \(Deb\)](#)

[Local Installer for Ubuntu20.04 cross-sbsa \(Deb\)](#)

[Local Installer for Ubuntu22.04 cross-sbsa \(Deb\)](#)

Local Installers for Red Hat (x86_64, armsbsa, Power architecture)

[Local Installer for RedHat/Centos 7.1 x86_64 \(RPM\)](#)

[Local Installer for RedHat/Centos 8.1 x86_64 \(RPM\)](#)

[Local Installer for RedHat/Centos 8.1 aarch64sbsa \(RPM\)](#)

[Local Installer for RedHat/Centos 8.1 PPC \(RPM\)](#)

[Local Installer for RedHat/Centos 9.1 x86_64 \(RPM\)](#)

[Local Installer for RedHat/Centos 9.1 aarch64sbsa \(RPM\)](#)

Download cuDNN v8.9.7 (December 5th, 2023), for CUDA 11.x

Download cuDNN v8.9.6 (November 1st, 2023), for CUDA 12.x

Download cuDNN v8.9.6 (November 1st, 2023), for CUDA 11.x

解压缩到 C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.1 替换

[cuDNN Archive | NVIDIA Developer](#)

话说 9.x.x 版本的 cuDNN 直接就是安装包了

cuDNN 安装好没有需要等 PyTorch

PyTorch 安装

Python

```
1 # 创建新环境（指定Python版本3.8或3.9）
2 conda create -n scene_classify python=3.9
3 conda activate scene_classify
```

```
C:\Users\qixua>conda env list
# conda environments:
#
base                D:\installbox\miniconda3
BASE                D:\installbox\miniconda3\envs\BASE
Fitness             D:\installbox\miniconda3\envs\Fitness
SET                 D:\installbox\miniconda3\envs\SET
deepLearn           D:\installbox\miniconda3\envs\deepLearn
scene_classify      D:\installbox\miniconda3\envs\scene_classify
```

新版本: [开始](#)

旧版本: [Previous PyTorch Versions](#)

v2.5.1

Conda

OSX

```
# conda
conda install pytorch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 -c pytorch
```

LINUX AND WINDOWS

```
# CUDA 11.8
conda install pytorch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 pytorch-cuda=11.8 -c py
# CUDA 12.1
conda install pytorch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 pytorch-cuda=12.1 -c pyt
# CUDA 12.4
conda install pytorch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 pytorch-cuda=12.4 -c pyt
# CPU Only
conda install pytorch==2.5.1 torchvision==0.20.1 torchaudio==2.5.1 cpuonly -c pytorch
```

检查

Algorithm
fitnessSystem
sceneClassify
temp.py
上课!
宿命的对决游戏开发
外部库
临时文件和控制台

```
1 import torch
2 print(torch.backends.cudnn.version())
3 # 能够正确返回
4 from torch.backends import cudnn # 若正常则静默
5 if torch.cuda.is_available():
6     print("GPU可用")
7     print("GPU数量:", torch.cuda.device_count())
8     print("GPU名称:", torch.cuda.get_device_name(0))
9     print("当前GPU索引:", torch.cuda.current_device())
10 else:
11     print("GPU不可用, 将使用CPU")
12 a = torch.tensor(1.)
13 print(cudnn.is_acceptable(a.cuda()))
14 # 若正常返回True
15
```

temp (1) ×

D:\installbox\miniconda3\envs\scene_classify\python.exe D:\superbox\python\pythonProject\sceneClassify\temp.py
90100
GPU可用
GPU数量: 1
GPU名称: NVIDIA GeForce RTX 2050
当前GPU索引: 0
True

进程已结束, 退出代码为 0

准备数据

Intel Image Classification

项目结构

Python

```
1 scene_classify/          # 项目根目录
2 |— data/                 # 存放数据集（你已准备好）
3 |— models/              # 存放模型定义
4 |— utils/               # 存放工具函数
5 |— experiments/         # 存放实验结果
6 |— checkpoints/         # 存放训练好的模型权重
7
8 powershell:
9 mkdir -p scene_classify/{data,models,utils,experiments,checkpoints}
10 cd scene_classify
```

实现步骤

1. 数据加载、预处理

由于是已经处理好的图像不涉及数据清晰、图像处理

Python

```
1 目标：
2 utils/data_loader.py    # 加载数据
3 utils/test_loader.py    # 测试加载模块
```

先写数据加载模块，因为这是所有实验的基础。实现一个灵活的数据加载器，支持：

- 划分训练集、验证集、测试集
- 可选择是否使用数据增强
- 返回类别名称映射

科研动机：

1. 为什么划分验证集？

验证集用于在训练过程中监控模型是否过拟合，帮助我们选择超参数（如早停）。如果不划分，直接用测试集调参会导致信息泄露，最终测试准确率不可信。

2. 为什么使用标准化（Normalization）？

图像像素值范围是 0~255，直接输入网络会导致梯度不稳定。标准化（减均值除方差）使数据分布接近标准正态，加速收敛。我们使用 ImageNet 的均值和标准差，因为这是通用的预训练统计量，即使我们不预训练，也能让数据处于合理范围。

3. 为什么数据增强？

深度学习模型需要大量数据，但实际数据有限。数据增强通过对训练图片进行随机变换（翻转、旋转、颜色抖动），生成更多样的样本，相当于扩充了数据集，提高泛化能力。

4. 为什么要自己写灵活的数据集类？

因为我们需要在训练集上应用增强，而在验证集上**不能**应用增强（保持数据纯净），同时要保证训练集和验证集样本互斥。PyTorch 的 `ImageFolder` 默认不支持这种分离，所以我们用自定义类实现。

2. 阶段一：CNN

Python

```
1 目标：
2 models/simple_cnn.py      # 模型
3 utils/train_eval.py      # 训练工具
4 stage1_train.py          # 训练脚本
```

1. 定义模型文件 `models/simple_cnn.py`
2. 编写训练脚本 `stage1_train.py`，包含训练、验证、测试和结果保存
3. 运行阶段一，观察结果

`simple_cnn.py`

- 这是最简单的 CNN 基线，用于后续对比。
- 为什么用 3 层卷积？因为图像尺寸不大（150x150），3 层卷积足以提取基本特征，且参数量适中。
- 全连接层降维到 256，最后输出 6 类。

结果

Python

```
1 使用设备: cuda
2 加载数据...
3 类别映射: {0: 'buildings', 1: 'forest', 2: 'glacier', 3: 'mountain', 4:
  'sea', 5: 'street'}
4 模型参数量: 10711878
5 开始训练...
6 Epoch 1/10: Train Loss: 1.0768, Acc: 0.5869 | Val Loss: 0.8285, Acc: 0.
  6901
7 Epoch 2/10: Train Loss: 0.6407, Acc: 0.7617 | Val Loss: 0.6209, Acc: 0.
  7720
8 Epoch 3/10: Train Loss: 0.5079, Acc: 0.8182 | Val Loss: 0.5555, Acc: 0.
  7912
9 Epoch 4/10: Train Loss: 0.4226, Acc: 0.8449 | Val Loss: 0.5846, Acc: 0.
  7962
10 Epoch 5/10: Train Loss: 0.3413, Acc: 0.8755 | Val Loss: 0.6004, Acc: 0.
  7969
11 Epoch 6/10: Train Loss: 0.2464, Acc: 0.9123 | Val Loss: 0.6178, Acc: 0.
  8126
12 Epoch 7/10: Train Loss: 0.1597, Acc: 0.9431 | Val Loss: 0.6993, Acc: 0.
  8123
13 Epoch 8/10: Train Loss: 0.0958, Acc: 0.9662 | Val Loss: 0.9312, Acc: 0.
  7880
14 Epoch 9/10: Train Loss: 0.0631, Acc: 0.9801 | Val Loss: 0.9184, Acc: 0.
  7959
15 Epoch 10/10: Train Loss: 0.0362, Acc: 0.9894 | Val Loss: 0.9463, Acc:
  0.8204
16 训练完成, 耗时: 462.41秒
17 在测试集上评估...
18 测试准确率: 0.8253
```

3. 阶段二：加入 BatchNorm 和 Dropout

在每个卷积层后添加批归一化（Batch Normalization），并在全连接层前添加 **Dropout**，然后重新训练

科研动机

- **BatchNorm** 的作用：

- 加速收敛：通过对每一层的输出进行标准化，使网络中间输出的分布更加稳定，避免梯度消失 / 爆炸，可以使用更大的学习率。
- 正则化效果：由于每个 mini-batch 的均值和方差有噪声，BN 带来轻微的正则化，有时可以替代 Dropout。

- **Dropout 的作用：**

- 随机丢弃部分神经元，强制网络学习冗余特征，防止过拟合。

- **为什么组合使用：**

- BN 主要解决内部协变量偏移，加速训练；Dropout 主要解决过拟合。二者互补，尤其在阶段一可能过拟合的情况下（训练损失下降而验证损失上升），加入 Dropout 能有效缓解。

Python

```
1 目标：
2 models/cnn_bn_dropout.py
3 stage2_train.py
```

cnn_bn_dropout.py

- 每个卷积层后紧跟一个 `BatchNorm2d` 层。
- 在第一个全连接层 `fc1` 的输出后、`fc2` 之前添加 `Dropout(0.5)`。

stage2_train.py

结果

Python

```
1 使用设备: cuda
2 加载数据...
3 类别映射: {0: 'buildings', 1: 'forest', 2: 'glacier', 3: 'mountain', 4:
  'sea', 5: 'street'}
4 模型参数量: 10712326
5 开始训练阶段二...
6 Epoch 1/10: Train Loss: 1.9402, Acc: 0.5238 | Val Loss: 0.9291, Acc: 0.
  6459
7 Epoch 2/10: Train Loss: 0.9892, Acc: 0.6125 | Val Loss: 0.8408, Acc: 0.
  7011
8 Epoch 3/10: Train Loss: 0.8790, Acc: 0.6614 | Val Loss: 0.7435, Acc: 0.
  7535
9 Epoch 4/10: Train Loss: 0.7829, Acc: 0.7106 | Val Loss: 0.6823, Acc: 0.
  7553
10 Epoch 5/10: Train Loss: 0.7183, Acc: 0.7337 | Val Loss: 0.6439, Acc: 0.
  7617
11 Epoch 6/10: Train Loss: 0.7006, Acc: 0.7391 | Val Loss: 0.6853, Acc: 0.
  7677
12 Epoch 7/10: Train Loss: 0.6549, Acc: 0.7565 | Val Loss: 0.6095, Acc: 0.
  7980
13 Epoch 8/10: Train Loss: 0.6209, Acc: 0.7748 | Val Loss: 0.6152, Acc: 0.
  7784
14 Epoch 9/10: Train Loss: 0.6135, Acc: 0.7736 | Val Loss: 0.5226, Acc: 0.
  8180
15 Epoch 10/10: Train Loss: 0.5822, Acc: 0.7805 | Val Loss: 0.5783, Acc:
  0.8016
16 训练完成, 耗时: 397.15秒
17 在测试集上评估...
18 测试准确率: 0.8297
```

阶段二结果（从 82.53% 到 82.97%）提升确实不明显，但这在科研中很正常——并非每个改进都能带来大幅提升，关键在于**理解原因**。可能的原因有：

- **基线已经较强**：你的 SimpleCNN 可能已经很好地拟合了数据，82.5% 对于这个数据集已经是合理基线（有些公开实现简单 CNN 也就 70% 多，但你的可能因为超参数、数据划分等原因更高）。
- **BN+Dropout 的作用**：BN 主要加速收敛，但在 10 个 epoch 内可能已经足够；Dropout 针对过拟合，如果你的模型没有严重过拟合（训练和验证曲线接近），那么 Dropout 的增益自

然有限。

- **数据集特性：**Intel Scene 分类任务可能比较简单，类间差异大，简单模型也能达到不错效果。

尽管如此，**0.4% 的提升仍然是正向的**，证明模块有效。在复试汇报时，你可以坦诚说明这一点，并强调**即使提升微小，也验证了 BN 和 Dropout 的正则化作用，防止了潜在的过拟合**（可以通过观察训练 / 验证曲线来佐证）。

4. 阶段三：加入数据增强

Python

- 1 目标：
- 2 stage3_train.py

Python

- 1 在 data_loader.py 中定义了增强变换：
- 2 python
- 3 transforms.RandomHorizontalFlip(p=0.5) # 随机水平翻转
- 4 transforms.RandomRotation(10) # 随机旋转 ±10 度
- 5 transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1) # 颜色抖动
- 6 这些变换对于场景分类是合理的：
- 7 翻转：自然照片可能左右镜像（如山脉、海洋）。
- 8 旋转：手持拍摄可能有轻微倾斜。
- 9 颜色抖动：模拟不同光照条件。
- 10 注意：验证集和测试集绝不应用数据增强，以保持评估的客观性。我们的 get_data_loaders 已经通过 train_augment 参数控制，训练集使用增强，验证 / 测试使用基础变换。

结果

Python

```
1 使用设备: cuda
2 加载数据 (启用增强) ...
3 类别映射: {0: 'buildings', 1: 'forest', 2: 'glacier', 3: 'mountain', 4:
  'sea', 5: 'street'}
4 模型参数量: 10712326
5 开始训练阶段三...
6 Epoch 1/10: Train Loss: 1.9588, Acc: 0.4460 | Val Loss: 1.0743, Acc: 0.
  5843
7 Epoch 2/10: Train Loss: 1.1179, Acc: 0.5481 | Val Loss: 0.9044, Acc: 0.
  6413
8 Epoch 3/10: Train Loss: 1.0351, Acc: 0.5891 | Val Loss: 0.8591, Acc: 0.
  6997
9 Epoch 4/10: Train Loss: 0.9350, Acc: 0.6254 | Val Loss: 0.7804, Acc: 0.
  7431
10 Epoch 5/10: Train Loss: 0.8921, Acc: 0.6605 | Val Loss: 0.6781, Acc: 0.
  7613
11 Epoch 6/10: Train Loss: 0.8678, Acc: 0.6728 | Val Loss: 0.7248, Acc: 0.
  7496
12 Epoch 7/10: Train Loss: 0.8413, Acc: 0.6729 | Val Loss: 0.9096, Acc: 0.
  6765
13 Epoch 8/10: Train Loss: 0.8159, Acc: 0.6928 | Val Loss: 0.6139, Acc: 0.
  7855
14 Epoch 9/10: Train Loss: 0.8006, Acc: 0.6948 | Val Loss: 0.5776, Acc: 0.
  7998
15 Epoch 10/10: Train Loss: 0.7692, Acc: 0.7056 | Val Loss: 0.6280, Acc:
  0.7845
16 训练完成, 耗时: 390.71秒
17 测试准确率: 0.8097
```

准确率从 82.97% 降到 80.97%，下降了 2 个百分点。这看起来反常，但**完全可以解释**，甚至能成为你复试中的一个亮点——展示你如何**诊断问题、提出假设并验证**。

可能原因诊断

1. 数据增强强度可能过大

- 我们使用的增强（随机翻转、旋转±10°、颜色抖动）可能对某些类别（如街道、建筑）引入了不真实的样本，导致模型学习到错误的特征。

- **验证：**可以查看训练 / 验证损失曲线。如果训练损失下降正常但验证损失震荡或上升，说明增强后的训练集与验证集分布差异大。

2. 训练 epoch 不足

- 数据增强增加了样本多样性，但每个 epoch 模型看到的都是变换后的图像，需要更多迭代才能充分学习。10 个 epoch 可能不够，模型还未收敛。
- **验证：**观察验证准确率曲线是否仍在上升。如果最后几个 epoch 验证准确率还在上升，说明需要更多 epoch。

3. 随机性影响

- 单次运行结果可能有波动。可以重复实验取平均，但为节省时间，我们可先调整参数。

4. 模型容量不足

- 模型可能无法从增强数据中提取足够泛化的特征，需要更复杂的结构（这正是阶段四要验证的）。

5. 阶段四：引入残差连接（简化 ResNet）

Python

```
1 models/resblock.py
2 models/simple_resnet.py
```

残差连接通过“跳跃连接”让梯度可以直接流向浅层，解决了深层网络梯度消失问题，使我们能训练更深的网络

结果

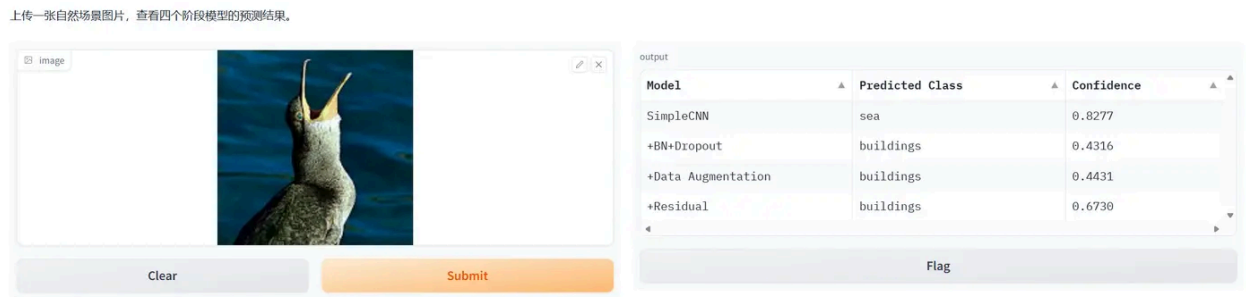
```
1 使用设备: cuda
2 加载数据 (启用增强) ...
3 类别映射: {0: 'buildings', 1: 'forest', 2: 'glacier', 3: 'mountain', 4:
  'sea', 5: 'street'}
4 模型参数量: 11184390
5 开始训练阶段四...
6 Epoch 1/10: Train Loss: 1.0667, Acc: 0.5738 | Val Loss: 1.6774, Acc: 0.
  4863
7 Epoch 2/10: Train Loss: 0.7463, Acc: 0.7233 | Val Loss: 0.7696, Acc: 0.
  7232
8 Epoch 3/10: Train Loss: 0.6185, Acc: 0.7698 | Val Loss: 0.9254, Acc: 0.
  6673
9 Epoch 4/10: Train Loss: 0.5609, Acc: 0.7943 | Val Loss: 0.6195, Acc: 0.
  7916
10 Epoch 5/10: Train Loss: 0.5350, Acc: 0.8052 | Val Loss: 0.5949, Acc: 0.
  7880
11 Epoch 6/10: Train Loss: 0.4968, Acc: 0.8195 | Val Loss: 0.5184, Acc: 0.
  8165
12 Epoch 7/10: Train Loss: 0.4531, Acc: 0.8397 | Val Loss: 0.4586, Acc: 0.
  8411
13 Epoch 8/10: Train Loss: 0.4340, Acc: 0.8441 | Val Loss: 0.4650, Acc: 0.
  8365
14 Epoch 9/10: Train Loss: 0.4188, Acc: 0.8520 | Val Loss: 0.4516, Acc: 0.
  8425
15 Epoch 10/10: Train Loss: 0.3981, Acc: 0.8580 | Val Loss: 0.4191, Acc:
  0.8579
16 训练完成, 耗时: 471.72秒
17 测试准确率: 0.8613
```

6. 汇总实验结果与分析

Python

```
1 目标：
2 collect_results.py      # 提取数据
3 plot_results.py        # 绘制对比柱状图
4 plot_curves.py          # 绘制训练曲线
5 models/ablation_study.py      # 消融实验：无残差的普通网络，层数相同
6 ablation_study.py      # 消融实验脚本。对每种消融设置训练5个epoch，记录测试准确率
7 plot_confusion.py      # 混淆矩阵
8 gradio_demo.py         # 加载四个模型并实现预测
9 # 会生成一个本地URL，可在浏览器打开测试。
pip install gradio
```

智能相册分类助手 - 模块化改进对比



发现这张“精心挑选”的图片，改进后的不如最开始的（但是相比之下残差效果最好）

7. 继续探索

可能的原因

1. 模型容量与过拟合

- **SimpleCNN（阶段一）**：参数量最小（约 1071 万），结构简单，可能只学到最显著的全局特征（如海水的蓝色、纹理）。对于这张图片，它正确识别为“sea”。
- **阶段二 / 三 / 四**：加入了 BN、Dropout、数据增强、残差连接，模型容量增大（阶段四约 1120 万），且训练时数据增强引入了更多变体。这可能导致模型关注了更细微的特征，但这些特征在测试图片上可能与“buildings”更相似。

关键点：复杂的模型在训练时可能过度拟合了训练集中的某种模式（例如“海边建筑”类样本中既有海水又有建筑），导致对“sea”类的判别边界变得保守，反而将海水 + 背景误判为“buildings”。

2. 数据增强的副作用

- 阶段三和阶段四使用了数据增强（随机翻转、旋转、颜色抖动）。如果增强策略过于强烈，可能使得训练集中的“sea”类样本被变换后看起来像“buildings”（例如旋转后海平线倾斜，颜色抖动后海水变灰，类似建筑颜色）。
- 模型被迫学习更鲁棒的特征，但也可能因此模糊了类别间的边界，导致对特定测试样本的误判。

3. 随机性与局部最优

- 各阶段模型独立训练，可能收敛到不同的局部最优。阶段一的随机初始化恰好对这张图片有利，而其他阶段的随机初始化不利。可以尝试多次运行取平均，但单次结果具有偶然性。

4. 类别不平衡或样本分布

- 检查训练集中“sea”和“buildings”的样本数量。如果“buildings”样本远多于“sea”，模型可能偏向预测“buildings”。阶段一的简单模型可能没学到这种偏向，而复杂模型学到了。
- 也可以查看混淆矩阵：阶段四的整体准确率 86% 高于阶段一的 82%，说明它在整体上更好，但可能在个别类别（如“sea”）上牺牲了精度。

验证猜想

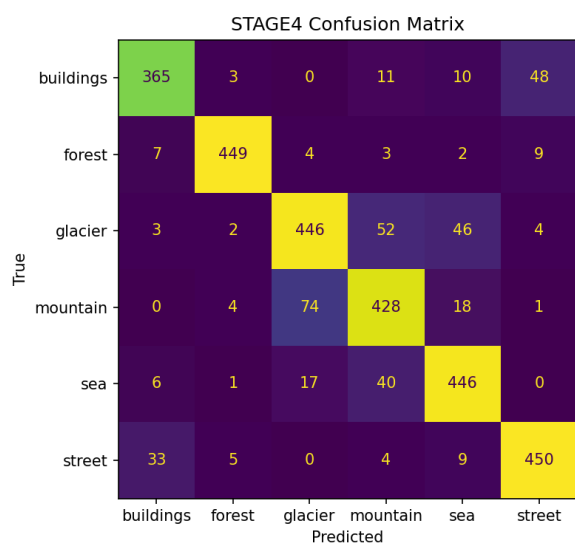
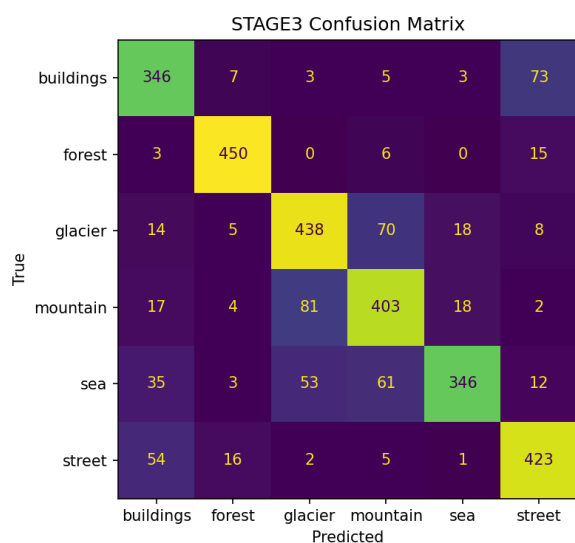
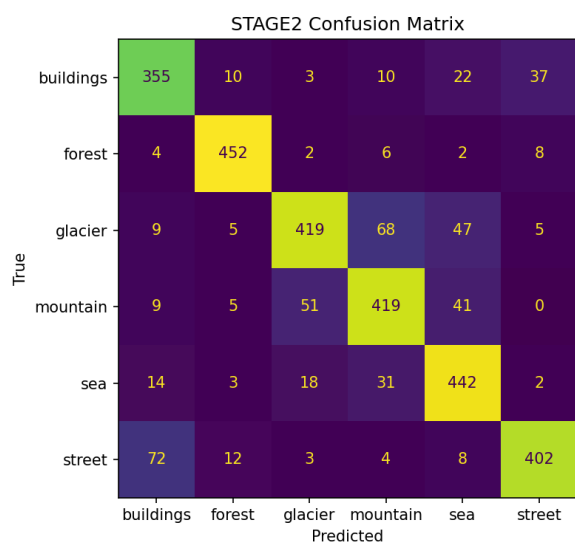
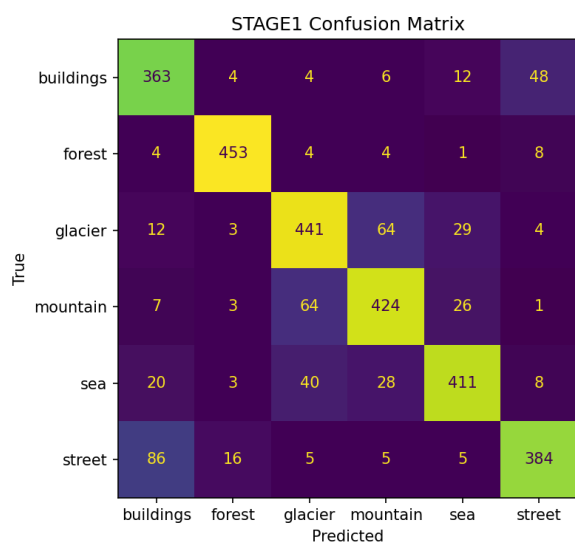
1. 统计整体性能，而非单张图片

你已经得到阶段四的测试准确率 86% > 阶段一的 82%，说明在**整个测试集**上，复杂模型确实更好。单张图片的反常并不能否定整体趋势。

建议：在 Demo 中展示时，可以多测试几张图片，并强调整体准确率的提升。同时，也可以展示混淆矩阵，说明哪些类别容易混淆。

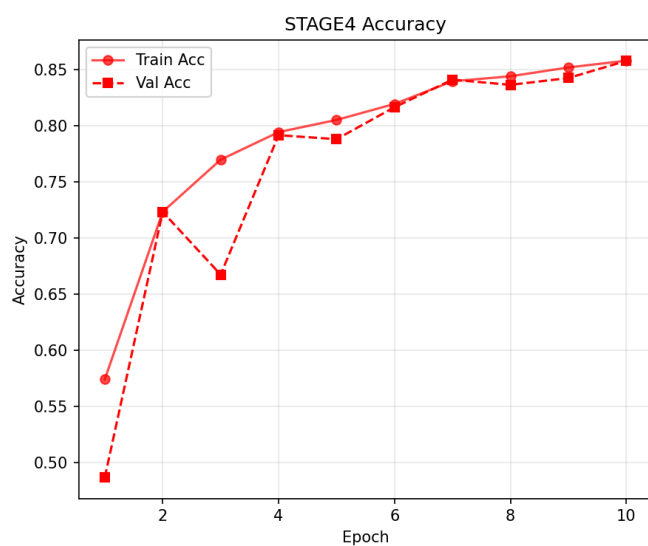
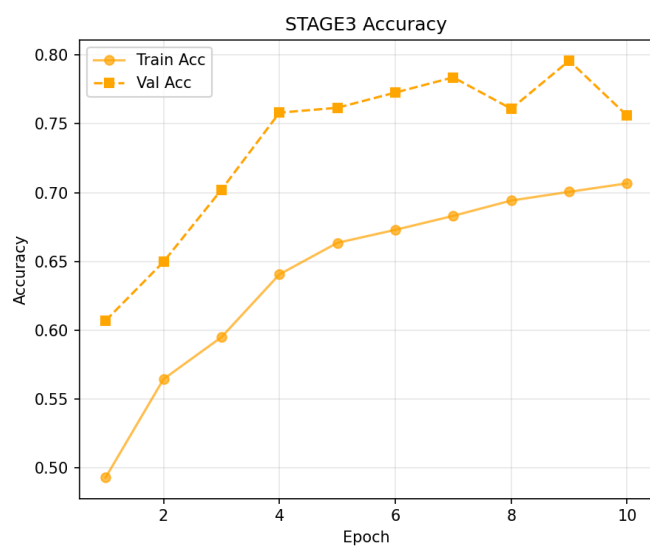
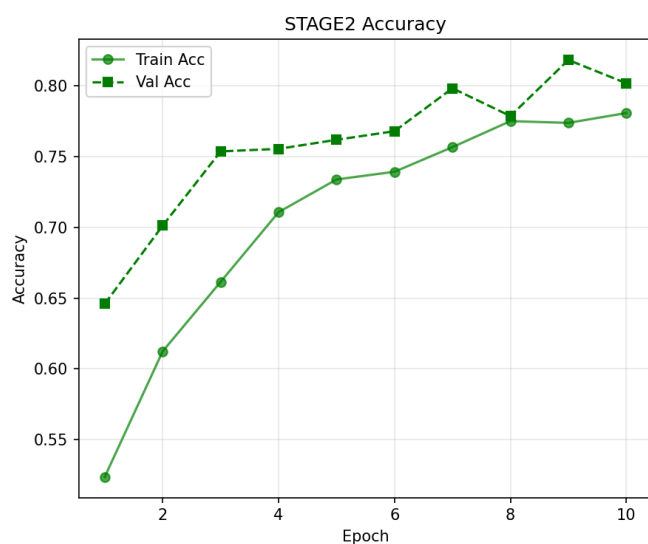
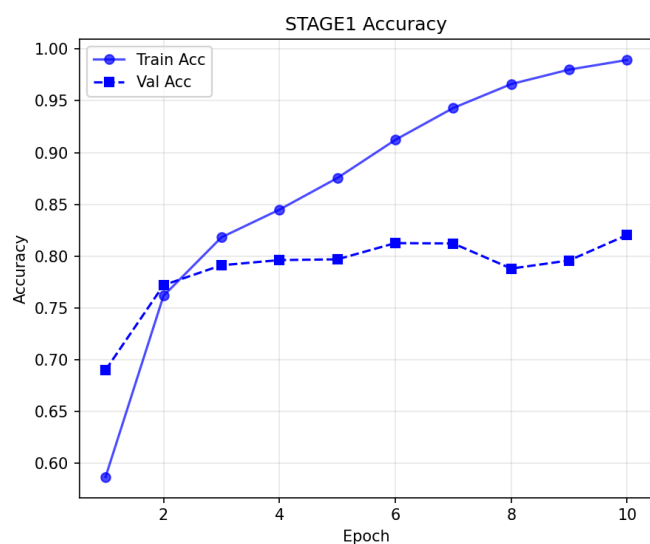
2. 分析混淆矩阵

运行 `plot_confusion.py`，查看阶段四在“sea”类上的召回率是否低于阶段一。如果阶段四的“sea”召回率下降，而“buildings”精度上升，说明模型牺牲了“sea”来换取整体准确率。



3. 检查训练曲线

- 绘制阶段四的训练 / 验证损失曲线，观察是否存在过拟合。如果验证损失在后期上升，说明模型过拟合，可能在测试集上泛化不佳。
- 阶段一的曲线可能更平稳，泛化能力更好。



说明：

横坐标：epoch (1-10)

纵坐标：准确率 (0-1)

每阶段有两条线：训练准确率（实线）和验证准确率（虚线）

理想曲线：两条线同步上升，最终平稳且差距小。

异常情况：

过拟合：训练准确率持续上升，验证准确率上升后下降（差距拉大）

欠拟合：两条线都偏低且上升缓慢

训练不稳定：准确率突然剧烈波动

1 阶段一 (SimpleCNN)

2 训练曲线：从 0.55 迅速跃升至 0.76 (第2 epoch)，然后近乎线性上升到 0.99。

3 验证曲线：从 0.69 跃升至 0.77，之后在 0.75~0.83 之间波动，无法继续提升。

4 解读：

5 训练准确率接近 100%，说明模型容量足以完全记住训练集。

6 验证准确率停滞且波动，表明模型过拟合严重——学到了训练集中的噪声，无法泛化到新数据。

7 最高验证准确率约 0.83，但稳定性差，这是简单 CNN 的典型特征：容易过拟合，泛化能力受限。

8

9 阶段二 (+BN+Dropout)

10 训练曲线：从 0.51 近乎对数增长，最终约 0.77。

11 验证曲线：始终高于训练，从 0.65 增长到 0.80，第 8 epoch 异常下降后第 9 回升。

12 解读：

13 BN 和 Dropout 起到了强正则化作用：训练时 Dropout 随机丢弃神经元，导致训练准确率偏低；验证时关闭 Dropout，因此准确率更高。

14 验证高于训练说明没有过拟合，泛化良好。

15 第 8 epoch 的下降可能是学习率或数据批次造成的临时波动，但很快恢复。

16 最终验证准确率约 0.80，略低于阶段一的最高点 (0.83)，但胜在稳定，且训练曲线仍在上升，延长训练可能进一步提升。

17

18 阶段三 (+数据增强)

19 训练曲线：从 0.49 对数增长到约 0.70。

20 验证曲线：始终高于训练，从 0.60 增长到 0.75，同样在第 8 epoch 出现波动。

21 解读：

22 数据增强增加了训练难度，使模型学习更鲁棒的特征，但也导致训练准确率更低（因为每张图都不同）。

23 验证高于训练，泛化能力好，但准确率低于阶段二，可能原因：

24 10 个 epoch 不足以充分学习增强后的数据。

25 增强策略可能过强，让类别边界模糊。

26 曲线仍在上升趋势，若延长训练有望超越阶段二。

27

28 阶段四 (+残差连接)

29 训练曲线：从 0.57 快速增长，随后趋缓，最终约 0.85。

30 验证曲线：位于训练曲线下方，呈锯齿形波动，但波动逐渐减小，与训练差距缩小。

31

32 解读：

33 残差网络容量更大，训练准确率明显高于阶段二/三。

34 验证准确率起初低于训练，但随着训练进行，差距逐渐收窄，说明过拟合得到控制，模型在充分利用容量的同时保持泛化。

35 锯齿形波动可能是由于学习率偏高或批次差异，但后期趋于稳定，表明模型收敛良好。

36 最终验证准确率可能接近 0.84~0.85，超过阶段一的最佳点，且稳定性更好。

37

38 综合结论

39 过拟合演进：

40 阶段一：严重过拟合（训练≈100%，验证≈83%）

41 阶段二/三：正则化有效，验证高于训练，但准确率上限受限于容量或训练时长

42 阶段四：容量与正则化平衡，训练≈85%，验证≈84%，过拟合最小

43

44 异常波动：

45 阶段二、三在第 8 epoch 同时出现验证下降，可能是数据加载或优化器造成的偶发事件，但恢复后不影响整体趋势。

46

47 最佳模型选择：

48 应基于验证准确率最高的 checkpoint，而非最后一轮。阶段四的第 5~7 epoch 可能已接近最优，最终测试准确率 86% 印证了这一点。

49

50 在复试中如何表述

51 “通过训练曲线，我深入分析了各阶段的学习动态：

52 阶段一的训练准确率直逼 100%，验证却停滞在 83%，暴露出严重过拟合。

53 加入 BN 和 Dropout 后，验证准确率反超训练，说明正则化生效，但受限于模型容量，准确率提升有限。

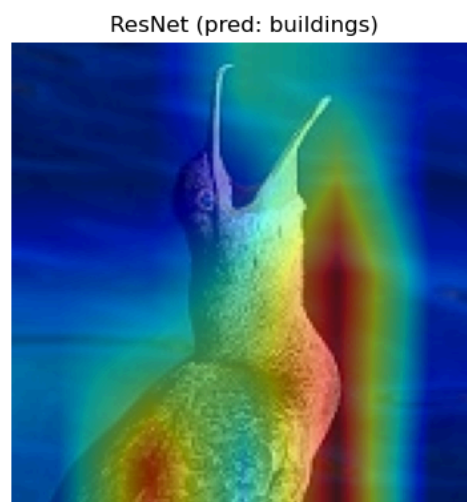
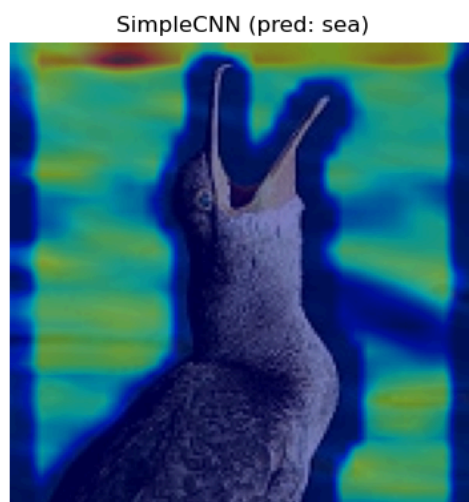
54 数据增强进一步抑制过拟合，但训练变慢，需要更多 epoch。

55 残差网络在保持高训练精度的同时，验证与训练差距不断缩小，最终在测试集上达到 86%，实现了容量与泛化的最佳平衡。

56 这些曲线不仅验证了各模块的作用，也让我学会了如何通过验证曲线诊断过拟合、选择早停点，为后续研究积累了经验。”

4. 对该图片进行可视化分析

- **Grad-CAM**：可以生成热力图，观察模型关注图像的哪些区域。阶段一可能关注海水区域，而阶段四可能关注了背景中的建筑轮廓，导致误判。

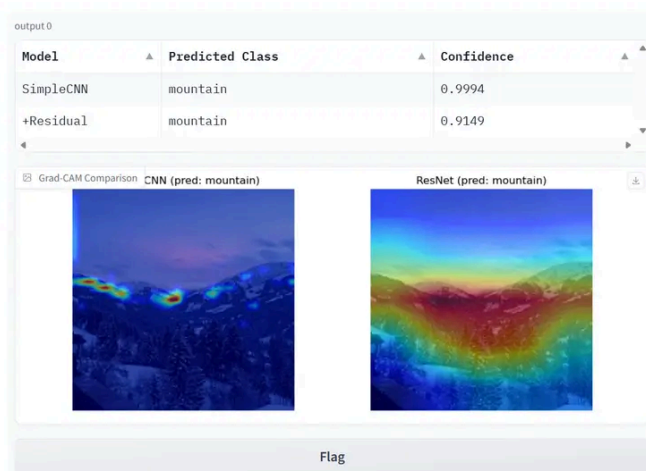
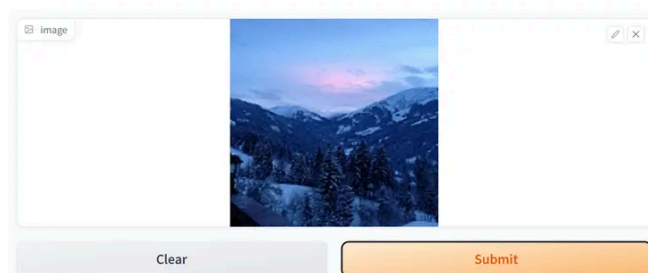


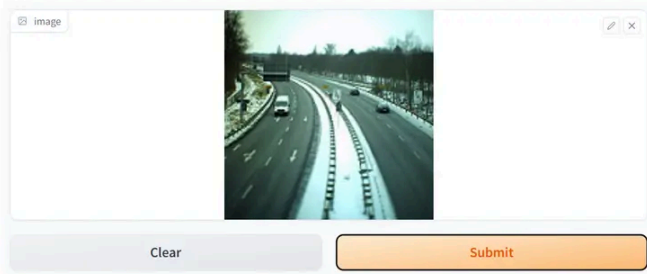
8. 添加热力图

Python

```
1 pip install opencv-python matplotlib
2 gradio_demo.py进行修改，加入热力图
3 新建gradio_demo_CAM.py
```

- Grad-CAM 利用目标类别的梯度，计算出最后一个卷积层每个通道的权重，然后加权求和得到热力图。
- 热力图中红色区域表示对分类贡献最大的区域，蓝色表示贡献小。
- 通过对比简单模型（SimpleCNN）和复杂模型（阶段四）的热力图，可以分析它们的关注点差异





output 0

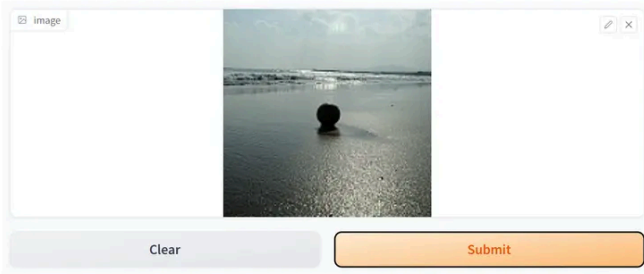
Model	Predicted Class	Confidence
SimpleCNN	street	0.9831
+Residual	buildings	0.5097

Grad-CAM Comparison

leCNN (pred: street)



ResNet (pred: buildings)

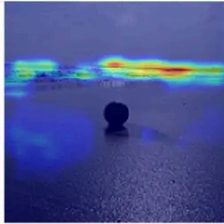


output 0

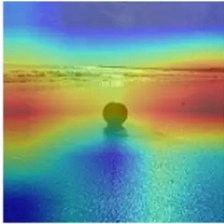
Model	Predicted Class	Confidence
SimpleCNN	sea	1.0000
+Residual	sea	0.9780

Grad-CAM Comparison

leCNN (pred: sea)

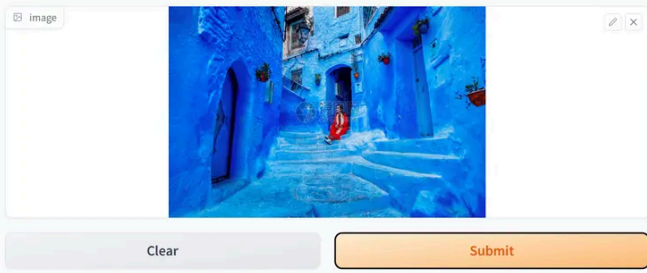


ResNet (pred: sea)



Flag

在网上随便找的图片：

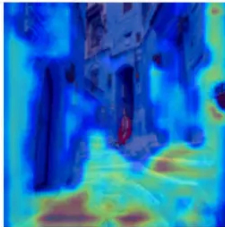


output 0

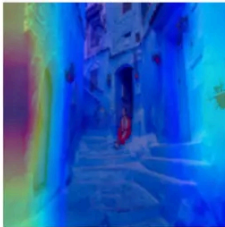
Model	Predicted Class	Confidence
SimpleCNN	sea	0.9406
+Residual	sea	0.3750

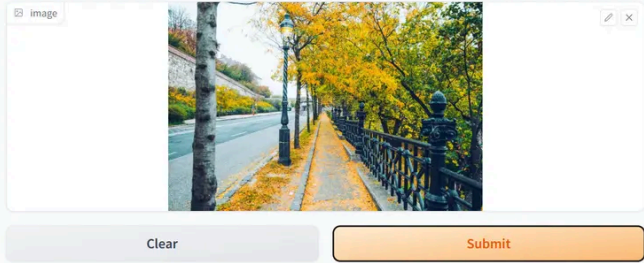
Grad-CAM Comparison

leCNN (pred: sea)



ResNet (pred: sea)





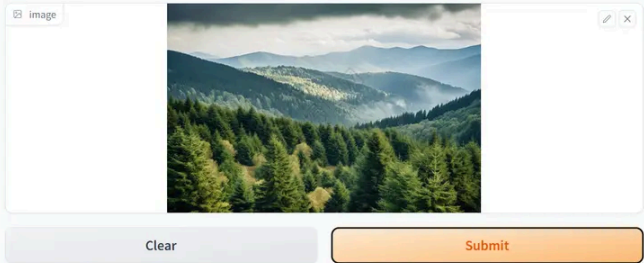
output 0

Model	Predicted Class	Confidence
SimpleCNN	street	0.5295
+Residual	street	0.5717

Grad-CAM Comparison

leCNN (pred: street)

ResNet (pred: street)



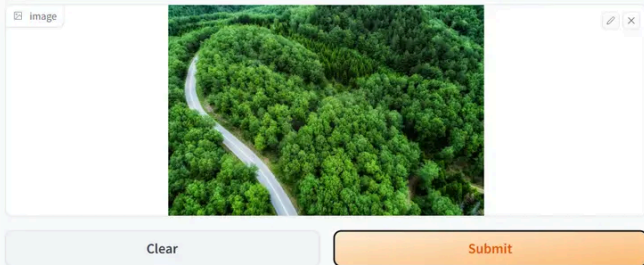
output 0

Model	Predicted Class	Confidence
SimpleCNN	mountain	0.8915
+Residual	mountain	0.9473

Grad-CAM Comparison

CNN (pred: mountain)

ResNet (pred: mountain)



output 0

Model	Predicted Class	Confidence
SimpleCNN	forest	0.9921
+Residual	forest	0.4198

Grad-CAM Comparison

leCNN (pred: forest)

ResNet (pred: forest)

- 1 “在对比实验的基础上，我还对个别样本进行了热力图分析。
- 2 有趣的是，虽然残差网络整体准确率更高，但在某些图片上，简单CNN的关注区域更广，预测也更准确。
- 3 例如这张图片（展示你的对比图），残差网络的热力图平滑且集中在建筑区域，却误判为建筑；而简单CNN关注了海水和天空，正确识别为海。
- 4 这说明：模块化改进并非在所有样本上都能带来提升。
- 5 残差网络倾向于学习更抽象、全局的特征，可能忽略了局部细节，导致对某些场景的误判。
- 6 而简单模型保留了更多纹理信息，反而在某些情况下更有效。
- 7 这启示我们：在实际应用中，可以结合多个模型的优势，或者根据场景选择合适模型。
- 8 未来工作可以探索模型融合或自适应选择策略。”

我在网络上随机找了一组照片，运行后发现在热力图上，残差网络可能关注的更为平滑范围比较大。但是在某些情况下，CNN 的关注范围更广，准确率也高于残差网络。并不绝对某个准确率更高

热力图观察结论

1. 为什么残差网络的热力图更平滑、范围更大？

- **感受野更大**：残差网络（如 SimpleResNet）层数更深，最后一个卷积层的感受野覆盖几乎整个图像，因此热力图通常覆盖更大区域。
- **特征抽象程度高**：深层网络提取的是高级语义特征（如“建筑轮廓”、“海面纹理”），这些特征在空间上更连续，导致热力图平滑。
- **残差连接的影响**：跳跃连接让梯度更容易传播，使得网络能够学习到更稳定的特征表示，热力图噪声更少，显得平滑。

2. 为什么简单 CNN 有时关注范围更广且准确率更高？

- **浅层特征**：简单 CNN 的最后一层卷积可能仍保留较多空间细节，关注区域可能更分散（例如同时关注海水和天空）。
- **过拟合或欠拟合**：对于某些特定样本，简单模型可能恰好“猜对”，而复杂模型因为训练时见过太多变体，反而对某些模式不敏感。
- **偶然性**：单次预测受随机初始化、数据增强等影响，存在波动

——“虽然阶段四已达到 86% 准确率，但仍存在优化空间。首先，从训练曲线看，阶段三的数据增强可能受益于更多 epoch，我计划延长训练并加入学习率衰减。其次，模型结构上可以引入注意力机制，如 SE 模块，进一步提升特征表达能力。此外，集成四个阶段的模型，利用它们的

互补性，也能小幅提升性能。最后，作为对比，迁移学习（如使用预训练 ResNet）可能达到更高准确率，但可能会掩盖模块化改进的贡献，因此更适合作为未来探索方向。这些改进都有文献支撑，且可逐步验证，体现了科研的持续迭代精神。”

上传至 GitHub

Python

```
1 pip list --format=freeze > requirements.txt      # 生成仅含包和版本的依赖文件
2 剩下的可以在GitHub Desktop操作
```

继续优化（贼心不死）

目前观察结果：

残差网络准确率对数上升，两条曲线的间距迅速减小

从混淆矩阵观察到的是，glacler 和 mountain、sea 容易混淆，buildings 和 street 容易混淆；对于 buildings 的识别准确率从 CNN 到残差网络没有提升。

阶段五：MixUp 数据增强

对于每个训练批次，随机选择两个样本 (x_i, y_i) 和 (x_j, y_j) ，按参数 $\lambda \sim \text{Beta}(\alpha, \alpha)$ 进行混合：

- 图像混合： $x' = \lambda x_i + (1 - \lambda)x_j$
- 标签混合： $y' = \lambda y_i + (1 - \lambda)y_j$ （软标签）

损失函数为交叉熵的加权和：

$$\mathcal{L} = \lambda \cdot \text{CE}(f(x'), y_i) + (1 - \lambda) \cdot \text{CE}(f(x'), y_j)$$

Plain Text

```
1 stage5_train_mixup.py
2 utils/train_eval.py      # 新建train_one_epoch_mixup方法
3 plot_stage5.py           # 混淆矩阵和训练曲线
4 evaluate_stage5.py       # 本阶段最佳训练模型观察
```

在阶段四（简化 ResNet）基础上，我们引入 **MixUp** 数据增强，以进一步提升模型泛化能力，尤其针对易混淆类别（冰川 vs 山 vs 海，建筑 vs 街道）。MixUp 通过随机混合两张图像及其标签，让模型学习特征之间的线性关系，能有效减少过拟合，增强对相似类别的区分能力。

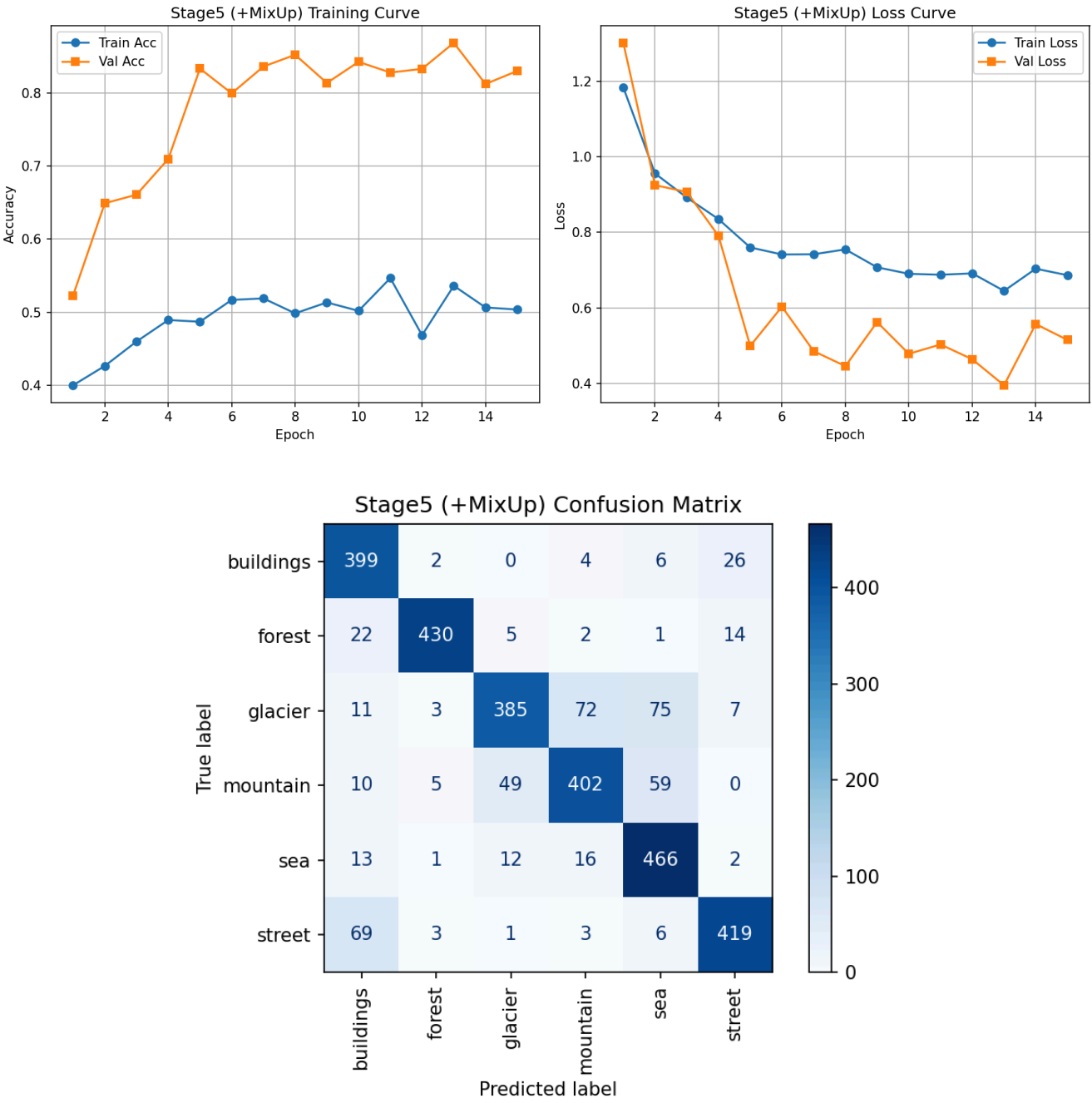
训练结果

Plain Text

```
1 使用设备: cuda
2 加载数据 (启用增强) ...
3 类别映射: {0: 'buildings', 1: 'forest', 2: 'glacier', 3: 'mountain', 4:
  'sea', 5: 'street'}
4 模型参数量: 11184390
5 开始训练阶段五 (+MixUp) ...
6 Epoch 1/15: Train Loss: 1.1832, Acc: 0.4001 | Val Loss: 1.3012, Acc: 0.
  5230
7 Epoch 2/15: Train Loss: 0.9556, Acc: 0.4266 | Val Loss: 0.9248, Acc: 0.
  6491
8 Epoch 3/15: Train Loss: 0.8921, Acc: 0.4600 | Val Loss: 0.9068, Acc: 0.
  6608
9 Epoch 4/15: Train Loss: 0.8350, Acc: 0.4894 | Val Loss: 0.7907, Acc: 0.
  7097
10 Epoch 5/15: Train Loss: 0.7601, Acc: 0.4870 | Val Loss: 0.4996, Acc: 0.
  8333
11 Epoch 6/15: Train Loss: 0.7415, Acc: 0.5169 | Val Loss: 0.6034, Acc: 0.
  7994
12 Epoch 7/15: Train Loss: 0.7421, Acc: 0.5189 | Val Loss: 0.4859, Acc: 0.
  8361
13 Epoch 8/15: Train Loss: 0.7549, Acc: 0.4985 | Val Loss: 0.4462, Acc: 0.
  8522
14 Epoch 9/15: Train Loss: 0.7075, Acc: 0.5134 | Val Loss: 0.5627, Acc: 0.
  8137
15 Epoch 10/15: Train Loss: 0.6904, Acc: 0.5019 | Val Loss: 0.4788, Acc:
  0.8425
16 Epoch 11/15: Train Loss: 0.6879, Acc: 0.5467 | Val Loss: 0.5035, Acc:
  0.8279
17 Epoch 12/15: Train Loss: 0.6915, Acc: 0.4686 | Val Loss: 0.4643, Acc:
  0.8329
18 Epoch 13/15: Train Loss: 0.6452, Acc: 0.5362 | Val Loss: 0.3956, Acc:
  0.8685
19 Epoch 14/15: Train Loss: 0.7038, Acc: 0.5065 | Val Loss: 0.5574, Acc:
  0.8123
20 Epoch 15/15: Train Loss: 0.6867, Acc: 0.5037 | Val Loss: 0.5155, Acc:
  0.8301
21 训练完成, 耗时: 1654.71秒
22 在测试集上评估...
```

23 测试准确率：0.8337

24 阶段五实验完成，结果已保存。



分析

- 训练准确率持续偏低（0.4-0.55），而验证准确率快速升至 0.83 以上，最高达到 0.8685。
- 验证损失远低于训练损失，且在第 13 个 epoch 达到最低 0.3956。
- 验证准确率从第 5 epoch 起就稳定在 0.83 以上，波动但总体向上。

MixUp 导致训练准确率低估：在训练过程中，我们使用混合图像和软标签，但计算准确率时仍使用原始硬标签，这会导致训练准确率被严重低估（因为模型预测的是混合图像，但目标标签

是原始标签)。这是正常现象，不代表模型训练差。

验证准确率真实反映模型性能：验证时没有 MixUp，使用原始图像和硬标签，因此验证准确率是可信的。

验证损失低于训练损失，说明模型在验证集上的预测与硬标签更吻合，这正是 MixUp 正则化的效果——使模型对未见数据更鲁棒。

所以需要测试集评估 13epoch 的模型，得到真实的测试准确率（已经保存的就是了）就是 0.8685

阶段六：注意力机制(SE)

- **Squeeze**：通过全局平均池化将每个通道的空间特征压缩为一个标量，得到通道描述符。
- **Excitation**：通过两个全连接层学习每个通道的权重（sigmoid 激活），实现通道间的依赖建模。
- **Scale**：将学习到的权重乘以原始特征图，实现特征重标定。

Plain Text

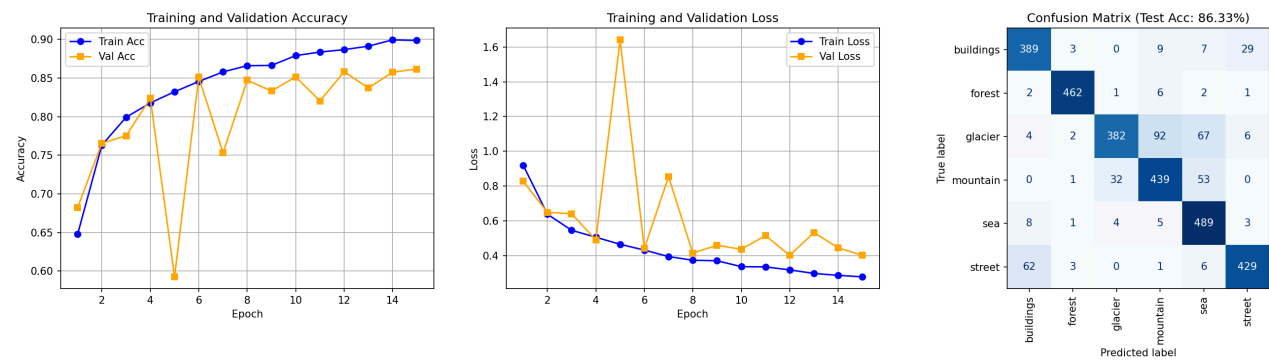
```
1 models/se_resnet.py          # 带SE模块的残差块和网络
2 stage6_train_se.py          # 基于阶段四的训练逻辑，但模型替换为SEResNet
3 plot_stage6.py              # 图
```

训练结果

Plain Text

```
1 使用设备: cuda
2 加载数据 (启用增强) ...
3 类别映射: {0: 'buildings', 1: 'forest', 2: 'glacier', 3: 'mountain', 4:
  'sea', 5: 'street'}
4 模型参数量: 11273470
5 开始训练阶段六 (+SE注意力) ...
6 Epoch 1/15: Train Loss: 0.9199, Acc: 0.6475 | Val Loss: 0.8271, Acc: 0.
  6822
7 Epoch 2/15: Train Loss: 0.6383, Acc: 0.7631 | Val Loss: 0.6492, Acc: 0.
  7656
8 Epoch 3/15: Train Loss: 0.5452, Acc: 0.7992 | Val Loss: 0.6406, Acc: 0.
  7752
9 Epoch 4/15: Train Loss: 0.5056, Acc: 0.8178 | Val Loss: 0.4896, Acc: 0.
  8237
10 Epoch 5/15: Train Loss: 0.4649, Acc: 0.8323 | Val Loss: 1.6412, Acc: 0.
  5924
11 Epoch 6/15: Train Loss: 0.4315, Acc: 0.8456 | Val Loss: 0.4407, Acc: 0.
  8511
12 Epoch 7/15: Train Loss: 0.3942, Acc: 0.8580 | Val Loss: 0.8539, Acc: 0.
  7531
13 Epoch 8/15: Train Loss: 0.3730, Acc: 0.8660 | Val Loss: 0.4144, Acc: 0.
  8472
14 Epoch 9/15: Train Loss: 0.3702, Acc: 0.8664 | Val Loss: 0.4592, Acc: 0.
  8333
15 Epoch 10/15: Train Loss: 0.3362, Acc: 0.8791 | Val Loss: 0.4362, Acc:
  0.8514
16 Epoch 11/15: Train Loss: 0.3348, Acc: 0.8837 | Val Loss: 0.5149, Acc:
  0.8204
17 Epoch 12/15: Train Loss: 0.3176, Acc: 0.8869 | Val Loss: 0.4012, Acc:
  0.8582
18 Epoch 13/15: Train Loss: 0.2972, Acc: 0.8913 | Val Loss: 0.5326, Acc:
  0.8372
19 Epoch 14/15: Train Loss: 0.2861, Acc: 0.8995 | Val Loss: 0.4442, Acc:
  0.8575
20 Epoch 15/15: Train Loss: 0.2778, Acc: 0.8987 | Val Loss: 0.4014, Acc:
  0.8618
21 训练完成, 耗时: 1218.90秒
22 在测试集上评估...
```

- 23 测试准确率：0.8633
- 24 阶段六实验完成，结果已保存。



相比阶段四几乎没有实质性提升，并且那几个类别的混淆问题没有得到根本的解决，无论是数据增强还是注意力机制都没有明显提升。当前训练策略或者模型容量可能接近瓶颈。需要更有效的技术增强对细微差异的敏感性并保持泛化能力

从图片上观察，我感觉学习率有问题并且可能有点过拟合，然后测试准确率比之前阶段四的ResNet 高，可能说明这个注意力机制是有效的，所以调整其他部分 have a try

阶段七：训练策略调优（解决过拟合与学习率问题）

1. 权重衰减（L2 正则化）：在优化器中添加 `weight_decay`，抑制过拟合。
2. 学习率余弦退火：使用 `CosineAnnealingLR` 让学习率周期性下降，帮助收敛。
3. 标签平滑（Label Smoothing）：软化硬标签，防止模型过于自信，提升泛化能力。
4. 梯度裁剪：防止梯度爆炸，稳定训练（可选）。

Plain Text

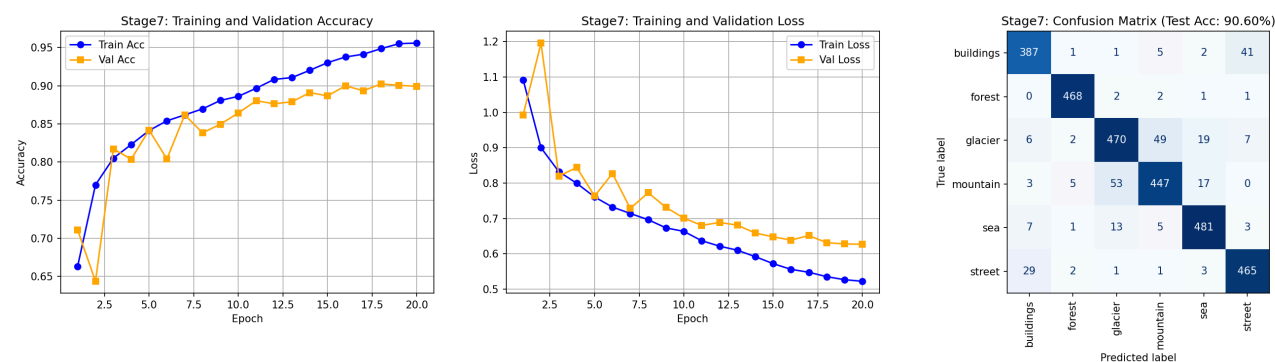
```
1 utils/losses.py          # 标签平滑损失函数
2 stage7_train_optim.py
3 plot_stage7.py
```

训练结果

Plain Text

```
1 使用设备: cuda
2 加载数据 (启用增强) ...
3 类别映射: {0: 'buildings', 1: 'forest', 2: 'glacier', 3: 'mountain', 4:
  'sea', 5: 'street'}
4 模型参数量: 11273470
5 开始训练阶段七 (优化策略) ...
6 Epoch 1/20: Train Loss: 1.0920, Acc: 0.6634 | Val Loss: 0.9929, Acc: 0.
  7111 | LR: 9.94e-04
7 Epoch 2/20: Train Loss: 0.8997, Acc: 0.7701 | Val Loss: 1.1959, Acc: 0.
  6437 | LR: 9.76e-04
8 Epoch 3/20: Train Loss: 0.8322, Acc: 0.8050 | Val Loss: 0.8199, Acc: 0.
  8172 | LR: 9.46e-04
9 Epoch 4/20: Train Loss: 0.7994, Acc: 0.8229 | Val Loss: 0.8440, Acc: 0.
  8037 | LR: 9.05e-04
10 Epoch 5/20: Train Loss: 0.7607, Acc: 0.8413 | Val Loss: 0.7631, Acc: 0.
  8418 | LR: 8.54e-04
11 Epoch 6/20: Train Loss: 0.7322, Acc: 0.8540 | Val Loss: 0.8271, Acc: 0.
  8041 | LR: 7.94e-04
12 Epoch 7/20: Train Loss: 0.7140, Acc: 0.8618 | Val Loss: 0.7282, Acc: 0.
  8618 | LR: 7.27e-04
13 Epoch 8/20: Train Loss: 0.6963, Acc: 0.8697 | Val Loss: 0.7728, Acc: 0.
  8383 | LR: 6.55e-04
14 Epoch 9/20: Train Loss: 0.6733, Acc: 0.8807 | Val Loss: 0.7312, Acc: 0.
  8497 | LR: 5.79e-04
15 Epoch 10/20: Train Loss: 0.6634, Acc: 0.8861 | Val Loss: 0.7008, Acc:
  0.8643 | LR: 5.01e-04
16 Epoch 11/20: Train Loss: 0.6369, Acc: 0.8967 | Val Loss: 0.6803, Acc:
  0.8803 | LR: 4.22e-04
17 Epoch 12/20: Train Loss: 0.6216, Acc: 0.9083 | Val Loss: 0.6886, Acc:
  0.8764 | LR: 3.46e-04
18 Epoch 13/20: Train Loss: 0.6101, Acc: 0.9107 | Val Loss: 0.6811, Acc:
  0.8789 | LR: 2.74e-04
19 Epoch 14/20: Train Loss: 0.5919, Acc: 0.9202 | Val Loss: 0.6585, Acc:
  0.8910 | LR: 2.07e-04
20 Epoch 15/20: Train Loss: 0.5721, Acc: 0.9303 | Val Loss: 0.6480, Acc:
  0.8867 | LR: 1.47e-04
21 Epoch 16/20: Train Loss: 0.5561, Acc: 0.9377 | Val Loss: 0.6387, Acc:
  0.8999 | LR: 9.64e-05
22 Epoch 17/20: Train Loss: 0.5478, Acc: 0.9413 | Val Loss: 0.6517, Acc:
```

0.8935 | LR: 5.54e-05
23 Epoch 18/20: Train Loss: 0.5354, Acc: 0.9486 | Val Loss: 0.6314, Acc: 0.9024 | LR: 2.54e-05
24 Epoch 19/20: Train Loss: 0.5268, Acc: 0.9551 | Val Loss: 0.6280, Acc: 0.9006 | LR: 7.15e-06
25 Epoch 20/20: Train Loss: 0.5223, Acc: 0.9559 | Val Loss: 0.6269, Acc: 0.8992 | LR: 1.00e-06
26 训练完成, 耗时: 3776.44秒
27 在测试集上评估...
28 测试准确率: 0.9060
29 阶段七实验完成, 结果已保存。



观察维度	描述
训练准确率	从 0.66 稳步上升至 0.956，损失从 1.09 降至 0.52，说明模型在训练集上学习充分。
验证准确率	从 0.71 波动上升，后期稳定在 0.89–0.90，最高达到 0.9024 （第 18 epoch）。
验证损失	从 0.99 降至 0.63 左右，与训练损失差距逐渐缩小，后期稳定。
学习率变化	余弦退火使学习率从 9.94e-4 平滑降至 1e-6，后期极低学习率帮助模型精细调优。
过拟合情况	训练与验证准确率最终差距约 5.6%，较阶段六（约 4%）略有扩大，但验证准确率稳定，未出现骤降，表明过拟合得到有效控制。

关键发现：第 14–20 epoch 验证准确率稳定在 0.89–0.90，且损失同步下降，说明模型已收敛。最佳验证准确率出现在第 18 epoch（0.9024），最终测试准确率 **90.60%** 与之吻合，证明了模型的泛化能力

- 冰川、山、海之间的混淆减少：**阶段七的优化策略有效提升了这些易混淆类别的区分度。

- **建筑 vs 街道的区分改善但有限：**仍有改进空间。
- **buildings 类别的准确率明显低于其他类别：**这是当前主要瓶颈。

阶段八：类别加权（针对性优化 buildings）

Python

```
1 stage8_train_weighted.py
2 plot_stage8.py
```

在数据加载后，统计训练集中每个类别的样本数，计算权重。常用公式： $\text{weight} = \frac{\text{总样本数}}{\text{类别数} * \text{该类样本数}}$ 或 $\text{weight} = 1 / \text{sqrt}(\text{class_count})$ 。

使用基于样本数倒数

训练结果

Python

```
1 使用设备: cuda
2 加载数据 (启用增强) ...
3 类别名称: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
4 类别权重: [1.059424  1.0139873  0.97584355  0.94046557  1.0440272  0.96625
18 ]
5 模型参数量: 11273470
6 开始训练阶段八 (类别加权) ...
7 Epoch 1/20: Train Loss: 0.9469, Acc: 0.6243 | Val Loss: 0.7298, Acc: 0.
7243 | LR: 9.94e-04
8 Epoch 2/20: Train Loss: 0.6451, Acc: 0.7634 | Val Loss: 0.6288, Acc: 0.
7645 | LR: 9.76e-04
9 Epoch 3/20: Train Loss: 0.5385, Acc: 0.8035 | Val Loss: 0.5316, Acc: 0.
8016 | LR: 9.46e-04
10 Epoch 4/20: Train Loss: 0.4959, Acc: 0.8189 | Val Loss: 0.6173, Acc: 0.
7880 | LR: 9.05e-04
11 Epoch 5/20: Train Loss: 0.4543, Acc: 0.8359 | Val Loss: 0.4725, Acc: 0.
8183 | LR: 8.54e-04
12 Epoch 6/20: Train Loss: 0.4092, Acc: 0.8555 | Val Loss: 0.6818, Acc: 0.
7588 | LR: 7.94e-04
13 Epoch 7/20: Train Loss: 0.3878, Acc: 0.8604 | Val Loss: 0.4175, Acc: 0.
8457 | LR: 7.27e-04
14 Epoch 8/20: Train Loss: 0.3579, Acc: 0.8700 | Val Loss: 0.4097, Acc: 0.
8539 | LR: 6.55e-04
15 Epoch 9/20: Train Loss: 0.3465, Acc: 0.8790 | Val Loss: 0.4967, Acc: 0.
8144 | LR: 5.79e-04
16 Epoch 10/20: Train Loss: 0.3109, Acc: 0.8853 | Val Loss: 0.4159, Acc:
0.8564 | LR: 5.01e-04
17 Epoch 11/20: Train Loss: 0.2865, Acc: 0.8972 | Val Loss: 0.3636, Acc:
0.8714 | LR: 4.22e-04
18 Epoch 12/20: Train Loss: 0.2622, Acc: 0.9065 | Val Loss: 0.3912, Acc:
0.8668 | LR: 3.46e-04
19 Epoch 13/20: Train Loss: 0.2452, Acc: 0.9092 | Val Loss: 0.3744, Acc:
0.8650 | LR: 2.74e-04
20 Epoch 14/20: Train Loss: 0.2244, Acc: 0.9178 | Val Loss: 0.3625, Acc:
0.8764 | LR: 2.07e-04
21 Epoch 15/20: Train Loss: 0.1988, Acc: 0.9254 | Val Loss: 0.3526, Acc:
0.8785 | LR: 1.47e-04
22 Epoch 16/20: Train Loss: 0.1803, Acc: 0.9344 | Val Loss: 0.3247, Acc:
0.8896 | LR: 9.64e-05
```

23 Epoch 17/20: Train Loss: 0.1658, Acc: 0.9397 | Val Loss: 0.3329, Acc: 0.8878 | LR: 5.54e-05

24 Epoch 18/20: Train Loss: 0.1487, Acc: 0.9459 | Val Loss: 0.3154, Acc: 0.8917 | LR: 2.54e-05

25 Epoch 19/20: Train Loss: 0.1421, Acc: 0.9492 | Val Loss: 0.3188, Acc: 0.8896 | LR: 7.15e-06

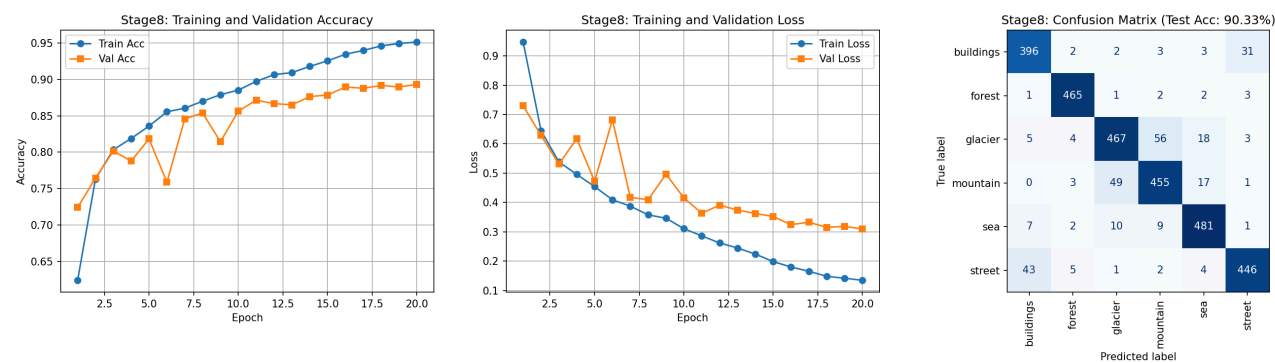
26 Epoch 20/20: Train Loss: 0.1350, Acc: 0.9512 | Val Loss: 0.3098, Acc: 0.8931 | LR: 1.00e-06

27 训练完成，耗时：1780.46秒

28 在测试集上评估...

29 测试准确率：0.9033

30 阶段八实验完成，结果已保存。



没有实质性的提升，和阶段七很接近

查找信息结论

- 参考 [Kaggle 公开 notebook](#) 中，用预训练 ResNet50 微调 20-30 epoch，加上 RandAugment 和 cosine annealing，测试准确率很容易达到 94%~96%。
- 若再结合集成，可稳定在 95% 以上。
- 如果坚持从零训练（不加载预训练），则 **95% 极为困难**，因为模型需要学习的特征量远大于数据量，可能已接近容量上限。但可以尝试更深的网络（如 ResNet101）配合更强的正则化，理论上可能接近 92%~93%，但 95% 希望渺茫。
- **Intel Image Classification** 是一个中等规模的数据集（6 类，约 1.4 万训练图像）。在 Kaggle 上，该数据集的最佳公开成绩大多在 **94%~96%** 左右，通常使用 **ImageNet 预训练模型**（如 ResNet50、EfficientNet）结合数据增强和微调获得。
- 从零训练（不加载预训练权重）能取得 90% 以上已经很优秀，因为模型需要从头学习通用视觉特征，而预训练模型已经具备强大的先验知识。

阶段八点五：集成学习

Python

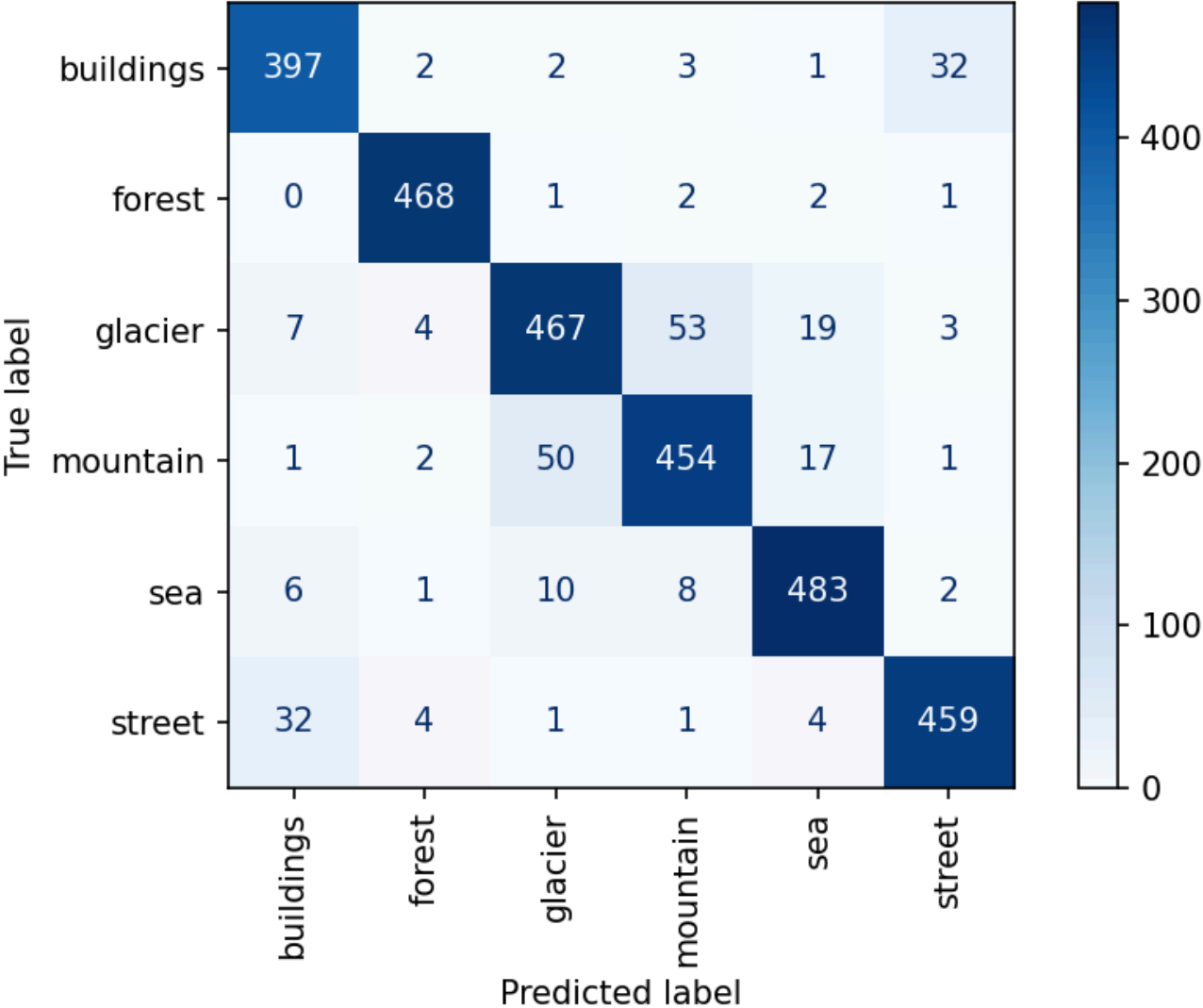
1 ensemble_stage7_8.py

结果

Python

```
1 集成模型测试准确率: 0.9093
2
3 分类报告:
4           precision    recall  f1-score   support
5
6  buildings      0.8962      0.9085      0.9023       437
7    forest      0.9730      0.9873      0.9801       474
8    glacier      0.8795      0.8445      0.8616       553
9    mountain      0.8714      0.8648      0.8681       525
10      sea      0.9183      0.9471      0.9324       510
11    street      0.9217      0.9162      0.9189       501
12
13    accuracy                    0.9093       3000
14  macro avg      0.9100      0.9114      0.9106       3000
15 weighted avg      0.9089      0.9093      0.9090       3000
```

Ensemble (Stage7+Stage8) Confusion Matrix (Acc: 90.93%)



当前训练可能尚未充分收敛：从阶段七和阶段八的日志看，验证准确率在最后几个 epoch 仍在上升（阶段七 epoch 20 的 0.8992 高于 epoch 18 的 0.9024。注意阶段七 epoch 18 是 0.9024，epoch 20 是 0.8992，略有下降，但 epoch 18 是最高点。阶段八 epoch 20 的 0.8931 是最高点）。这表明模型可能还有潜力，尤其如果学习率已降到极低（1e-6），后期微调仍有空间。

基于阶段七的成功配置（SE-ResNet、权重衰减、标签平滑、余弦退火），我们将总训练轮数增加到 **40 epoch**，并引入 **早停机制**（patience=7），在验证准确率连续 7 个 epoch 不创新高时自动停止训练，避免过拟合并捕捉最佳模型。

阶段九：更长 epoch + 早停（我要燃尽了）

Python

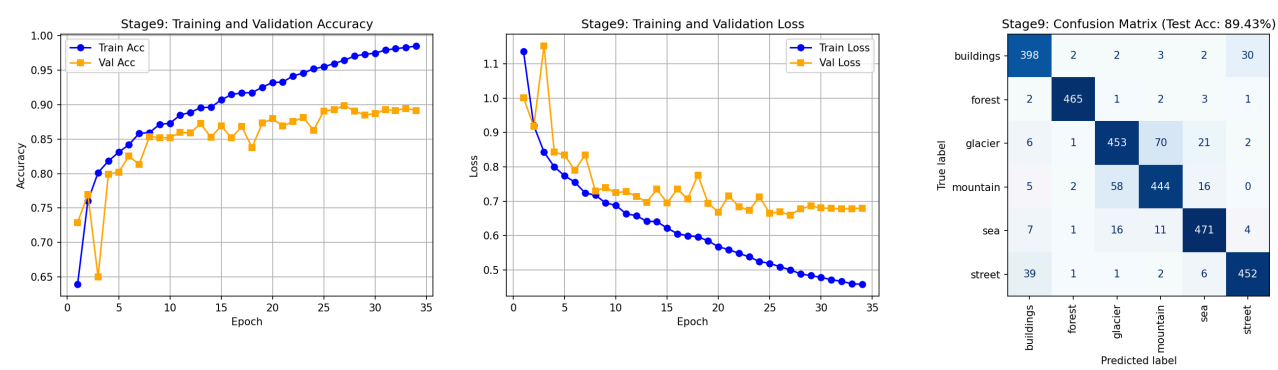
```
1 stage9_train_extended.py
2 plot_stage9.py
```

训练结果

```
1 使用设备: cuda
2 加载数据 (启用增强) ...
3 类别映射: {0: 'buildings', 1: 'forest', 2: 'glacier', 3: 'mountain', 4:
  'sea', 5: 'street'}
4 模型参数量: 11273470
5 开始训练阶段九 (延长训练 + 早停) ...
6 Epoch 1/40: Train Loss: 1.1359, Acc: 0.6392 | Val Loss: 1.0014, Acc: 0.
  7289 | LR: 9.98e-04
7   -> 新的最佳验证准确率: 0.7289
8 Epoch 2/40: Train Loss: 0.9203, Acc: 0.7602 | Val Loss: 0.9172, Acc: 0.
  7699 | LR: 9.94e-04
9   -> 新的最佳验证准确率: 0.7699
10 Epoch 3/40: Train Loss: 0.8429, Acc: 0.8015 | Val Loss: 1.1516, Acc: 0.
  6498 | LR: 9.86e-04
11   -> 连续 1 个epoch未提升
12 Epoch 4/40: Train Loss: 0.7999, Acc: 0.8181 | Val Loss: 0.8432, Acc: 0.
  7991 | LR: 9.76e-04
13   -> 新的最佳验证准确率: 0.7991
14 Epoch 5/40: Train Loss: 0.7739, Acc: 0.8309 | Val Loss: 0.8341, Acc: 0.
  8019 | LR: 9.62e-04
15   -> 新的最佳验证准确率: 0.8019
16 Epoch 6/40: Train Loss: 0.7556, Acc: 0.8419 | Val Loss: 0.7906, Acc: 0.
  8254 | LR: 9.46e-04
17   -> 新的最佳验证准确率: 0.8254
18 Epoch 7/40: Train Loss: 0.7240, Acc: 0.8584 | Val Loss: 0.8340, Acc: 0.
  8133 | LR: 9.26e-04
19   -> 连续 1 个epoch未提升
20 Epoch 8/40: Train Loss: 0.7175, Acc: 0.8594 | Val Loss: 0.7303, Acc: 0.
  8536 | LR: 9.05e-04
21   -> 新的最佳验证准确率: 0.8536
22 Epoch 9/40: Train Loss: 0.6950, Acc: 0.8715 | Val Loss: 0.7392, Acc: 0.
  8522 | LR: 8.80e-04
23   -> 连续 1 个epoch未提升
24 Epoch 10/40: Train Loss: 0.6879, Acc: 0.8729 | Val Loss: 0.7247, Acc:
  0.8522 | LR: 8.54e-04
25   -> 连续 2 个epoch未提升
26 Epoch 11/40: Train Loss: 0.6634, Acc: 0.8850 | Val Loss: 0.7282, Acc:
  0.8596 | LR: 8.25e-04
27   -> 新的最佳验证准确率: 0.8596
```

28 Epoch 12/40: Train Loss: 0.6578, Acc: 0.8888 | Val Loss: 0.7142, Acc: 0.8593 | LR: 7.94e-04
29 -> 连续 1 个epoch未提升
30 Epoch 13/40: Train Loss: 0.6416, Acc: 0.8958 | Val Loss: 0.6971, Acc: 0.8725 | LR: 7.61e-04
31 -> 新的最佳验证准确率: 0.8725
32 Epoch 14/40: Train Loss: 0.6403, Acc: 0.8963 | Val Loss: 0.7347, Acc: 0.8525 | LR: 7.27e-04
33 -> 连续 1 个epoch未提升
34 Epoch 15/40: Train Loss: 0.6217, Acc: 0.9069 | Val Loss: 0.6950, Acc: 0.8693 | LR: 6.92e-04
35 -> 连续 2 个epoch未提升
36 Epoch 16/40: Train Loss: 0.6053, Acc: 0.9148 | Val Loss: 0.7352, Acc: 0.8518 | LR: 6.55e-04
37 -> 连续 3 个epoch未提升
38 Epoch 17/40: Train Loss: 0.5995, Acc: 0.9171 | Val Loss: 0.7068, Acc: 0.8682 | LR: 6.17e-04
39 -> 连续 4 个epoch未提升
40 Epoch 18/40: Train Loss: 0.5965, Acc: 0.9175 | Val Loss: 0.7760, Acc: 0.8379 | LR: 5.79e-04
41 -> 连续 5 个epoch未提升
42 Epoch 19/40: Train Loss: 0.5844, Acc: 0.9253 | Val Loss: 0.6939, Acc: 0.8732 | LR: 5.40e-04
43 -> 新的最佳验证准确率: 0.8732
44 Epoch 20/40: Train Loss: 0.5671, Acc: 0.9324 | Val Loss: 0.6681, Acc: 0.8796 | LR: 5.00e-04
45 -> 新的最佳验证准确率: 0.8796
46 Epoch 21/40: Train Loss: 0.5593, Acc: 0.9328 | Val Loss: 0.7154, Acc: 0.8689 | LR: 4.61e-04
47 -> 连续 1 个epoch未提升
48 Epoch 22/40: Train Loss: 0.5480, Acc: 0.9418 | Val Loss: 0.6841, Acc: 0.8757 | LR: 4.22e-04
49 -> 连续 2 个epoch未提升
50 Epoch 23/40: Train Loss: 0.5379, Acc: 0.9458 | Val Loss: 0.6729, Acc: 0.8814 | LR: 3.84e-04
51 -> 新的最佳验证准确率: 0.8814
52 Epoch 24/40: Train Loss: 0.5242, Acc: 0.9522 | Val Loss: 0.7128, Acc: 0.8628 | LR: 3.46e-04
53 -> 连续 1 个epoch未提升
54 Epoch 25/40: Train Loss: 0.5192, Acc: 0.9549 | Val Loss: 0.6646, Acc: 0.8906 | LR: 3.09e-04

55 -> 新的最佳验证准确率: 0.8906
56 Epoch 26/40: Train Loss: 0.5089, Acc: 0.9597 | Val Loss: 0.6690, Acc: 0.8928 | LR: 2.74e-04
57 -> 新的最佳验证准确率: 0.8928
58 Epoch 27/40: Train Loss: 0.5004, Acc: 0.9646 | Val Loss: 0.6595, Acc: 0.8988 | LR: 2.40e-04
59 -> 新的最佳验证准确率: 0.8988
60 Epoch 28/40: Train Loss: 0.4883, Acc: 0.9706 | Val Loss: 0.6775, Acc: 0.8906 | LR: 2.07e-04
61 -> 连续 1 个epoch未提升
62 Epoch 29/40: Train Loss: 0.4835, Acc: 0.9732 | Val Loss: 0.6860, Acc: 0.8849 | LR: 1.76e-04
63 -> 连续 2 个epoch未提升
64 Epoch 30/40: Train Loss: 0.4781, Acc: 0.9748 | Val Loss: 0.6801, Acc: 0.8871 | LR: 1.47e-04
65 -> 连续 3 个epoch未提升
66 Epoch 31/40: Train Loss: 0.4717, Acc: 0.9793 | Val Loss: 0.6790, Acc: 0.8928 | LR: 1.21e-04
67 -> 连续 4 个epoch未提升
68 Epoch 32/40: Train Loss: 0.4665, Acc: 0.9813 | Val Loss: 0.6773, Acc: 0.8917 | LR: 9.64e-05
69 -> 连续 5 个epoch未提升
70 Epoch 33/40: Train Loss: 0.4599, Acc: 0.9832 | Val Loss: 0.6782, Acc: 0.8945 | LR: 7.46e-05
71 -> 连续 6 个epoch未提升
72 Epoch 34/40: Train Loss: 0.4581, Acc: 0.9851 | Val Loss: 0.6796, Acc: 0.8917 | LR: 5.54e-05
73 -> 连续 7 个epoch未提升
74 早停触发! 在第 34 个epoch停止训练
75 训练完成, 耗时: 2387.60秒
76 在测试集上评估...
77 测试准确率: 0.8943
78 阶段九实验完成, 结果已保存。



阶段九再次！出现了过拟合现象：

- **训练准确率持续上升：**从 epoch 20 的 0.9324 一直攀升到 epoch 34 的 0.9851，几乎接近完美拟合训练数据。
- **验证准确率停滞：**在 epoch 27 达到峰值 0.8988 后，后续 epoch 的验证准确率在 0.89 附近波动，不再提升，甚至略有下降。
- **差距扩大：**训练与验证准确率之差从早期的几个百分点扩大到后期接近 10%（训练 0.985 vs 验证 0.89），这是典型的过拟合信号——模型开始记忆训练集中的噪声，而无法泛化到新数据。

早停机制在连续 7 个 epoch 验证准确率未创新高时触发，保存了 epoch 27 的最佳模型（验证准确率 0.8988）。最终测试准确率为 0.8943，低于阶段七的 0.9060，说明延长训练并未带来提升，反而因为过拟合导致泛化能力略有下降

阶段十：强正则化 + 先进增强

```
Python

1 stage10_train_advanced.py
2 plot_stage10.py
```

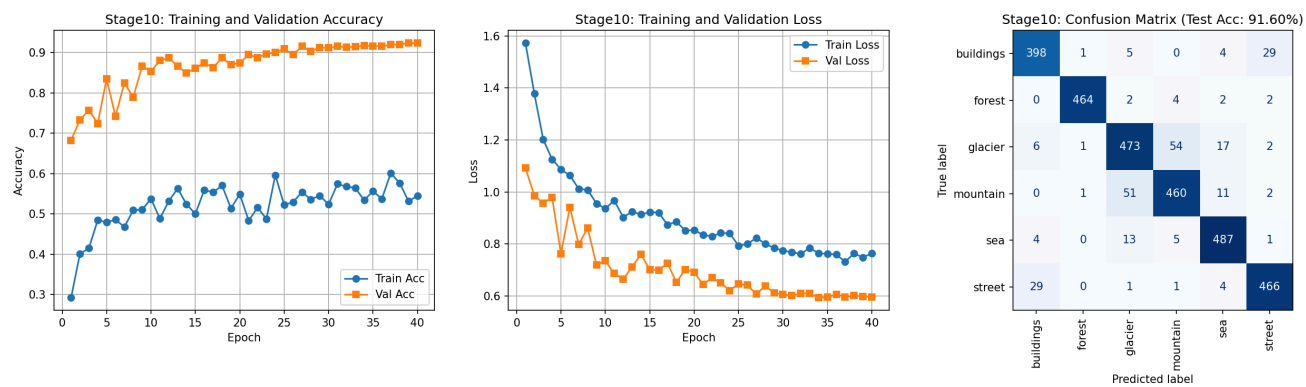
训练结果

```
1 使用设备: cuda
2 加载数据 (RandAugment + 弱化基础增强) ...
3 类别名称: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
4 模型参数量: 11273470
5 开始训练阶段十 (强正则化 + 先进增强) ...
6 Epoch 1/40: Train Loss: 1.5726, Acc: 0.2923 | Val Loss: 1.0924, Acc: 0.6826 | LR: 2.08e-04
7   -> 新的最佳验证准确率: 0.6826
8 Epoch 2/40: Train Loss: 1.3780, Acc: 0.4009 | Val Loss: 0.9860, Acc: 0.7335 | LR: 4.06e-04
9   -> 新的最佳验证准确率: 0.7335
10 Epoch 3/40: Train Loss: 1.2018, Acc: 0.4147 | Val Loss: 0.9577, Acc: 0.7570 | LR: 6.04e-04
11   -> 新的最佳验证准确率: 0.7570
12 Epoch 4/40: Train Loss: 1.1257, Acc: 0.4844 | Val Loss: 0.9789, Acc: 0.7243 | LR: 8.02e-04
13   -> 连续 1 个epoch未提升
14 Epoch 5/40: Train Loss: 1.0869, Acc: 0.4792 | Val Loss: 0.7622, Acc: 0.8347 | LR: 1.00e-03
15   -> 新的最佳验证准确率: 0.8347
16 Epoch 6/40: Train Loss: 1.0637, Acc: 0.4856 | Val Loss: 0.9416, Acc: 0.7417 | LR: 9.98e-04
17   -> 连续 1 个epoch未提升
18 Epoch 7/40: Train Loss: 1.0119, Acc: 0.4676 | Val Loss: 0.7978, Acc: 0.8251 | LR: 9.92e-04
19   -> 连续 2 个epoch未提升
20 Epoch 8/40: Train Loss: 1.0084, Acc: 0.5090 | Val Loss: 0.8618, Acc: 0.7887 | LR: 9.82e-04
21   -> 连续 3 个epoch未提升
22 Epoch 9/40: Train Loss: 0.9554, Acc: 0.5102 | Val Loss: 0.7195, Acc: 0.8668 | LR: 9.68e-04
23   -> 新的最佳验证准确率: 0.8668
24 Epoch 10/40: Train Loss: 0.9363, Acc: 0.5375 | Val Loss: 0.7351, Acc: 0.8536 | LR: 9.51e-04
25   -> 连续 1 个epoch未提升
26 Epoch 11/40: Train Loss: 0.9677, Acc: 0.4879 | Val Loss: 0.6866, Acc: 0.8803 | LR: 9.29e-04
27   -> 新的最佳验证准确率: 0.8803
28 Epoch 12/40: Train Loss: 0.9028, Acc: 0.5317 | Val Loss: 0.6653, Acc:
```

0.8867 | LR: 9.05e-04
29 -> 新的最佳验证准确率: 0.8867
30 Epoch 13/40: Train Loss: 0.9239, Acc: 0.5629 | Val Loss: 0.7108, Acc:
0.8664 | LR: 8.77e-04
31 -> 连续 1 个epoch未提升
32 Epoch 14/40: Train Loss: 0.9138, Acc: 0.5239 | Val Loss: 0.7604, Acc:
0.8489 | LR: 8.46e-04
33 -> 连续 2 个epoch未提升
34 Epoch 15/40: Train Loss: 0.9225, Acc: 0.5001 | Val Loss: 0.7018, Acc:
0.8618 | LR: 8.12e-04
35 -> 连续 3 个epoch未提升
36 Epoch 16/40: Train Loss: 0.9209, Acc: 0.5594 | Val Loss: 0.7001, Acc:
0.8742 | LR: 7.76e-04
37 -> 连续 4 个epoch未提升
38 Epoch 17/40: Train Loss: 0.8737, Acc: 0.5535 | Val Loss: 0.7254, Acc:
0.8625 | LR: 7.37e-04
39 -> 连续 5 个epoch未提升
40 Epoch 18/40: Train Loss: 0.8861, Acc: 0.5704 | Val Loss: 0.6528, Acc:
0.8871 | LR: 6.97e-04
41 -> 新的最佳验证准确率: 0.8871
42 Epoch 19/40: Train Loss: 0.8512, Acc: 0.5136 | Val Loss: 0.7020, Acc:
0.8703 | LR: 6.55e-04
43 -> 连续 1 个epoch未提升
44 Epoch 20/40: Train Loss: 0.8534, Acc: 0.5490 | Val Loss: 0.6921, Acc:
0.8746 | LR: 6.12e-04
45 -> 连续 2 个epoch未提升
46 Epoch 21/40: Train Loss: 0.8352, Acc: 0.4831 | Val Loss: 0.6441, Acc:
0.8945 | LR: 5.68e-04
47 -> 新的最佳验证准确率: 0.8945
48 Epoch 22/40: Train Loss: 0.8294, Acc: 0.5156 | Val Loss: 0.6706, Acc:
0.8878 | LR: 5.23e-04
49 -> 连续 1 个epoch未提升
50 Epoch 23/40: Train Loss: 0.8423, Acc: 0.4876 | Val Loss: 0.6507, Acc:
0.8960 | LR: 4.78e-04
51 -> 新的最佳验证准确率: 0.8960
52 Epoch 24/40: Train Loss: 0.8410, Acc: 0.5962 | Val Loss: 0.6212, Acc:
0.9006 | LR: 4.33e-04
53 -> 新的最佳验证准确率: 0.9006
54 Epoch 25/40: Train Loss: 0.7933, Acc: 0.5220 | Val Loss: 0.6465, Acc:
0.9092 | LR: 3.89e-04
55 -> 新的最佳验证准确率: 0.9092

56 Epoch 26/40: Train Loss: 0.8001, Acc: 0.5293 | Val Loss: 0.6418, Acc: 0.8945 | LR: 3.46e-04
57 -> 连续 1 个epoch未提升
58 Epoch 27/40: Train Loss: 0.8237, Acc: 0.5541 | Val Loss: 0.6073, Acc: 0.9156 | LR: 3.04e-04
59 -> 新的最佳验证准确率: 0.9156
60 Epoch 28/40: Train Loss: 0.8010, Acc: 0.5359 | Val Loss: 0.6393, Acc: 0.9031 | LR: 2.64e-04
61 -> 连续 1 个epoch未提升
62 Epoch 29/40: Train Loss: 0.7848, Acc: 0.5454 | Val Loss: 0.6115, Acc: 0.9127 | LR: 2.25e-04
63 -> 连续 2 个epoch未提升
64 Epoch 30/40: Train Loss: 0.7745, Acc: 0.5236 | Val Loss: 0.6071, Acc: 0.9116 | LR: 1.89e-04
65 -> 连续 3 个epoch未提升
66 Epoch 31/40: Train Loss: 0.7689, Acc: 0.5742 | Val Loss: 0.6012, Acc: 0.9156 | LR: 1.55e-04
67 -> 连续 4 个epoch未提升
68 Epoch 32/40: Train Loss: 0.7619, Acc: 0.5677 | Val Loss: 0.6106, Acc: 0.9134 | LR: 1.24e-04
69 -> 连续 5 个epoch未提升
70 Epoch 33/40: Train Loss: 0.7847, Acc: 0.5644 | Val Loss: 0.6100, Acc: 0.9145 | LR: 9.64e-05
71 -> 连续 6 个epoch未提升
72 Epoch 34/40: Train Loss: 0.7651, Acc: 0.5340 | Val Loss: 0.5939, Acc: 0.9170 | LR: 7.17e-05
73 -> 新的最佳验证准确率: 0.9170
74 Epoch 35/40: Train Loss: 0.7614, Acc: 0.5562 | Val Loss: 0.5953, Acc: 0.9163 | LR: 5.05e-05
75 -> 连续 1 个epoch未提升
76 Epoch 36/40: Train Loss: 0.7600, Acc: 0.5375 | Val Loss: 0.6066, Acc: 0.9159 | LR: 3.29e-05
77 -> 连续 2 个epoch未提升
78 Epoch 37/40: Train Loss: 0.7318, Acc: 0.6009 | Val Loss: 0.5968, Acc: 0.9195 | LR: 1.90e-05
79 -> 新的最佳验证准确率: 0.9195
80 Epoch 38/40: Train Loss: 0.7634, Acc: 0.5767 | Val Loss: 0.6016, Acc: 0.9202 | LR: 9.03e-06
81 -> 新的最佳验证准确率: 0.9202
82 Epoch 39/40: Train Loss: 0.7491, Acc: 0.5312 | Val Loss: 0.5990, Acc: 0.9234 | LR: 3.01e-06

83 -> 新的最佳验证准确率: 0.9234
84 Epoch 40/40: Train Loss: 0.7638, Acc: 0.5447 | Val Loss: 0.5956, Acc: 0.9238 | LR: 1.00e-06
85 -> 新的最佳验证准确率: 0.9238
86 训练完成, 耗时: 2778.58秒
87 在测试集上评估...
88 测试准确率: 0.9160
89 阶段十实验完成, 结果已保存。



- 最佳验证准确率: 92.38% (epoch 40)
 - 测试准确率: 91.60% (远超阶段七的 90.60% 和阶段九的 89.43%)
 - 训练过程:
 - 训练准确率维持在 50-60% (MixUp 导致), 验证准确率稳步提升至 92%+, 两者差距合理。
 - 验证损失持续下降, 无过拟合迹象 (训练损失同步下降但较高, 是 MixUp 正常现象)。
 - 早停未触发, 说明模型一直受益于更长的训练和精细的余弦退火。
- 强正则化: Dropout(0.3) + 更高 weight decay (5e-4) 有效抑制了过拟合, 使验证准确率在后期持续创新高。
 - 先进数据增强: RandAugment + 弱化基础增强 + MixUp 的组合极大地丰富了数据多样性, 迫使模型学习更鲁棒的特征。
 - 学习率策略: Warmup + 余弦退火让模型在早期稳定, 后期精细调优。

阶段十一：十基础上更长 epoch

阶段十 40epoch 验证损失仍在下降, 将 epoch 增加到 80, 早停 patience=10

Python

```
1 stage11_train_advanced.py  
2 plot_stage11.py
```

训练结果

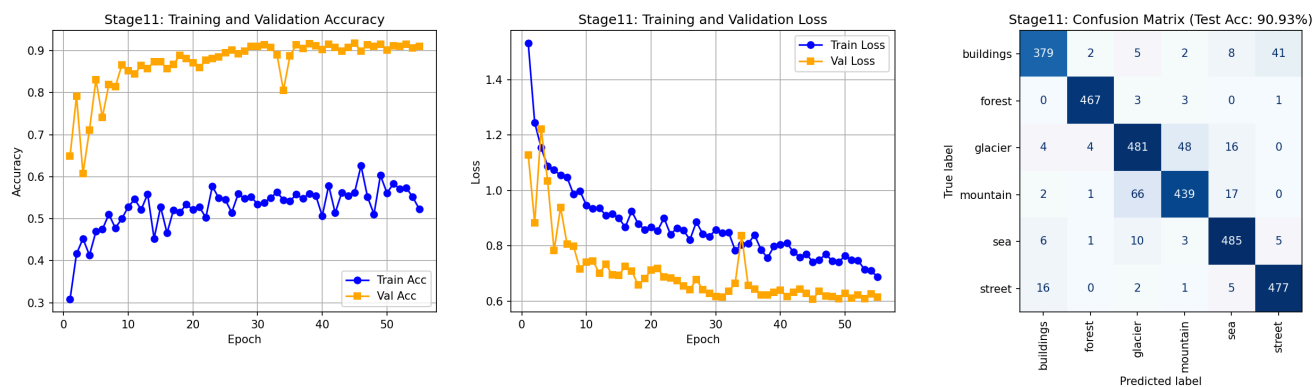
```
1 使用设备: cuda
2 加载数据 (RandAugment + 弱化基础增强) ...
3 类别名称: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
4 模型参数量: 11273470
5 开始训练阶段十一 (80 epoch + 早停10) ...
6 Epoch 1/80: Train Loss: 1.5318, Acc: 0.3084 | Val Loss: 1.1281, Acc: 0.
  6494 | LR: 2.08e-04
7   -> 新的最佳验证准确率: 0.6494
8 Epoch 2/80: Train Loss: 1.2451, Acc: 0.4164 | Val Loss: 0.8831, Acc: 0.
  7912 | LR: 4.06e-04
9   -> 新的最佳验证准确率: 0.7912
10 Epoch 3/80: Train Loss: 1.1543, Acc: 0.4526 | Val Loss: 1.2229, Acc: 0.
  6078 | LR: 6.04e-04
11   -> 连续 1 个epoch未提升
12 Epoch 4/80: Train Loss: 1.0877, Acc: 0.4129 | Val Loss: 1.0349, Acc: 0.
  7107 | LR: 8.02e-04
13   -> 连续 2 个epoch未提升
14 Epoch 5/80: Train Loss: 1.0739, Acc: 0.4696 | Val Loss: 0.7828, Acc: 0.
  8311 | LR: 1.00e-03
15   -> 新的最佳验证准确率: 0.8311
16 Epoch 6/80: Train Loss: 1.0547, Acc: 0.4745 | Val Loss: 0.9393, Acc: 0.
  7414 | LR: 1.00e-03
17   -> 连续 1 个epoch未提升
18 Epoch 7/80: Train Loss: 1.0477, Acc: 0.5107 | Val Loss: 0.8073, Acc: 0.
  8194 | LR: 9.98e-04
19   -> 连续 2 个epoch未提升
20 Epoch 8/80: Train Loss: 0.9868, Acc: 0.4779 | Val Loss: 0.7991, Acc: 0.
  8140 | LR: 9.96e-04
21   -> 连续 3 个epoch未提升
22 Epoch 9/80: Train Loss: 0.9982, Acc: 0.4997 | Val Loss: 0.7160, Acc: 0.
  8657 | LR: 9.93e-04
23   -> 新的最佳验证准确率: 0.8657
24 Epoch 10/80: Train Loss: 0.9461, Acc: 0.5283 | Val Loss: 0.7415, Acc:
  0.8522 | LR: 9.89e-04
25   -> 连续 1 个epoch未提升
26 Epoch 11/80: Train Loss: 0.9344, Acc: 0.5464 | Val Loss: 0.7460, Acc:
  0.8447 | LR: 9.84e-04
27   -> 连续 2 个epoch未提升
28 Epoch 12/80: Train Loss: 0.9367, Acc: 0.5215 | Val Loss: 0.7007, Acc:
```

0.8646 | LR: 9.79e-04
29 -> 连续 3 个epoch未提升
30 Epoch 13/80: Train Loss: 0.9106, Acc: 0.5587 | Val Loss: 0.7345, Acc:
0.8575 | LR: 9.72e-04
31 -> 连续 4 个epoch未提升
32 Epoch 14/80: Train Loss: 0.9157, Acc: 0.4527 | Val Loss: 0.6947, Acc:
0.8739 | LR: 9.65e-04
33 -> 新的最佳验证准确率: 0.8739
34 Epoch 15/80: Train Loss: 0.9009, Acc: 0.5280 | Val Loss: 0.6938, Acc:
0.8732 | LR: 9.57e-04
35 -> 连续 1 个epoch未提升
36 Epoch 16/80: Train Loss: 0.8673, Acc: 0.4658 | Val Loss: 0.7261, Acc:
0.8571 | LR: 9.48e-04
37 -> 连续 2 个epoch未提升
38 Epoch 17/80: Train Loss: 0.9256, Acc: 0.5210 | Val Loss: 0.7098, Acc:
0.8678 | LR: 9.38e-04
39 -> 连续 3 个epoch未提升
40 Epoch 18/80: Train Loss: 0.8795, Acc: 0.5150 | Val Loss: 0.6585, Acc:
0.8892 | LR: 9.28e-04
41 -> 新的最佳验证准确率: 0.8892
42 Epoch 19/80: Train Loss: 0.8592, Acc: 0.5346 | Val Loss: 0.6818, Acc:
0.8817 | LR: 9.17e-04
43 -> 连续 1 个epoch未提升
44 Epoch 20/80: Train Loss: 0.8687, Acc: 0.5211 | Val Loss: 0.7119, Acc:
0.8710 | LR: 9.05e-04
45 -> 连续 2 个epoch未提升
46 Epoch 21/80: Train Loss: 0.8549, Acc: 0.5277 | Val Loss: 0.7189, Acc:
0.8593 | LR: 8.92e-04
47 -> 连续 3 个epoch未提升
48 Epoch 22/80: Train Loss: 0.9007, Acc: 0.5028 | Val Loss: 0.6875, Acc:
0.8774 | LR: 8.79e-04
49 -> 连续 4 个epoch未提升
50 Epoch 23/80: Train Loss: 0.8412, Acc: 0.5770 | Val Loss: 0.6840, Acc:
0.8810 | LR: 8.65e-04
51 -> 连续 5 个epoch未提升
52 Epoch 24/80: Train Loss: 0.8648, Acc: 0.5493 | Val Loss: 0.6739, Acc:
0.8846 | LR: 8.50e-04
53 -> 连续 6 个epoch未提升
54 Epoch 25/80: Train Loss: 0.8568, Acc: 0.5452 | Val Loss: 0.6551, Acc:
0.8949 | LR: 8.35e-04
55 -> 新的最佳验证准确率: 0.8949

56 Epoch 26/80: Train Loss: 0.8223, Acc: 0.5147 | Val Loss: 0.6429, Acc: 0.9017 | LR: 8.19e-04
57 -> 新的最佳验证准确率: 0.9017
58 Epoch 27/80: Train Loss: 0.8862, Acc: 0.5589 | Val Loss: 0.6782, Acc: 0.8931 | LR: 8.02e-04
59 -> 连续 1 个epoch未提升
60 Epoch 28/80: Train Loss: 0.8424, Acc: 0.5481 | Val Loss: 0.6426, Acc: 0.8992 | LR: 7.86e-04
61 -> 连续 2 个epoch未提升
62 Epoch 29/80: Train Loss: 0.8331, Acc: 0.5519 | Val Loss: 0.6290, Acc: 0.9106 | LR: 7.68e-04
63 -> 新的最佳验证准确率: 0.9106
64 Epoch 30/80: Train Loss: 0.8585, Acc: 0.5349 | Val Loss: 0.6173, Acc: 0.9106 | LR: 7.50e-04
65 -> 连续 1 个epoch未提升
66 Epoch 31/80: Train Loss: 0.8466, Acc: 0.5387 | Val Loss: 0.6149, Acc: 0.9141 | LR: 7.32e-04
67 -> 新的最佳验证准确率: 0.9141
68 Epoch 32/80: Train Loss: 0.8481, Acc: 0.5494 | Val Loss: 0.6358, Acc: 0.9074 | LR: 7.13e-04
69 -> 连续 1 个epoch未提升
70 Epoch 33/80: Train Loss: 0.7840, Acc: 0.5632 | Val Loss: 0.6648, Acc: 0.8906 | LR: 6.94e-04
71 -> 连续 2 个epoch未提升
72 Epoch 34/80: Train Loss: 0.8039, Acc: 0.5446 | Val Loss: 0.8382, Acc: 0.8058 | LR: 6.75e-04
73 -> 连续 3 个epoch未提升
74 Epoch 35/80: Train Loss: 0.8080, Acc: 0.5420 | Val Loss: 0.6579, Acc: 0.8871 | LR: 6.55e-04
75 -> 连续 4 个epoch未提升
76 Epoch 36/80: Train Loss: 0.8393, Acc: 0.5579 | Val Loss: 0.6437, Acc: 0.9145 | LR: 6.35e-04
77 -> 新的最佳验证准确率: 0.9145
78 Epoch 37/80: Train Loss: 0.7859, Acc: 0.5479 | Val Loss: 0.6229, Acc: 0.9049 | LR: 6.15e-04
79 -> 连续 1 个epoch未提升
80 Epoch 38/80: Train Loss: 0.7566, Acc: 0.5601 | Val Loss: 0.6234, Acc: 0.9163 | LR: 5.94e-04
81 -> 新的最佳验证准确率: 0.9163
82 Epoch 39/80: Train Loss: 0.7982, Acc: 0.5550 | Val Loss: 0.6330, Acc: 0.9116 | LR: 5.73e-04

83 -> 连续 1 个epoch未提升
84 Epoch 40/80: Train Loss: 0.8038, Acc: 0.5067 | Val Loss: 0.6393, Acc: 0.9024 | LR: 5.53e-04
85 -> 连续 2 个epoch未提升
86 Epoch 41/80: Train Loss: 0.8098, Acc: 0.5790 | Val Loss: 0.6166, Acc: 0.9156 | LR: 5.32e-04
87 -> 连续 3 个epoch未提升
88 Epoch 42/80: Train Loss: 0.7781, Acc: 0.5135 | Val Loss: 0.6322, Acc: 0.9081 | LR: 5.11e-04
89 -> 连续 4 个epoch未提升
90 Epoch 43/80: Train Loss: 0.7596, Acc: 0.5616 | Val Loss: 0.6448, Acc: 0.8995 | LR: 4.90e-04
91 -> 连续 5 个epoch未提升
92 Epoch 44/80: Train Loss: 0.7708, Acc: 0.5550 | Val Loss: 0.6292, Acc: 0.9077 | LR: 4.69e-04
93 -> 连续 6 个epoch未提升
94 Epoch 45/80: Train Loss: 0.7421, Acc: 0.5623 | Val Loss: 0.6074, Acc: 0.9177 | LR: 4.48e-04
95 -> 新的最佳验证准确率: 0.9177
96 Epoch 46/80: Train Loss: 0.7485, Acc: 0.6261 | Val Loss: 0.6363, Acc: 0.8995 | LR: 4.28e-04
97 -> 连续 1 个epoch未提升
98 Epoch 47/80: Train Loss: 0.7703, Acc: 0.5520 | Val Loss: 0.6184, Acc: 0.9141 | LR: 4.07e-04
99 -> 连续 2 个epoch未提升
100 Epoch 48/80: Train Loss: 0.7460, Acc: 0.5107 | Val Loss: 0.6178, Acc: 0.9109 | LR: 3.86e-04
101 -> 连续 3 个epoch未提升
102 Epoch 49/80: Train Loss: 0.7410, Acc: 0.6032 | Val Loss: 0.6095, Acc: 0.9159 | LR: 3.66e-04
103 -> 连续 4 个epoch未提升
104 Epoch 50/80: Train Loss: 0.7654, Acc: 0.5614 | Val Loss: 0.6282, Acc: 0.9020 | LR: 3.46e-04
105 -> 连续 5 个epoch未提升
106 Epoch 51/80: Train Loss: 0.7499, Acc: 0.5830 | Val Loss: 0.6116, Acc: 0.9120 | LR: 3.26e-04
107 -> 连续 6 个epoch未提升
108 Epoch 52/80: Train Loss: 0.7468, Acc: 0.5712 | Val Loss: 0.6222, Acc: 0.9106 | LR: 3.07e-04
109 -> 连续 7 个epoch未提升
110 Epoch 53/80: Train Loss: 0.7154, Acc: 0.5733 | Val Loss: 0.6094, Acc:

0.9156 | LR: 2.88e-04
111 -> 连续 8 个epoch未提升
112 Epoch 54/80: Train Loss: 0.7100, Acc: 0.5514 | Val Loss: 0.6276, Acc:
0.9067 | LR: 2.69e-04
113 -> 连续 9 个epoch未提升
114 Epoch 55/80: Train Loss: 0.6883, Acc: 0.5229 | Val Loss: 0.6145, Acc:
0.9109 | LR: 2.51e-04
115 -> 连续 10 个epoch未提升
116 早停触发! 在第 55 个epoch停止训练
117 训练完成, 耗时: 6579.87秒
118 在测试集上评估...
119 测试准确率: 0.9093
120 阶段十一实验完成, 结果已保存。



- **最佳验证准确率: 91.77%** (epoch 45)
- **测试准确率: 90.93%**
- **早停触发:** 第 55 epoch, 连续 10 个 epoch 未创新高

与阶段十 (验证 92.38%, 测试 91.60%) 相比, 阶段十一在验证和测试上均略有下降。这说明:

1. **延长训练并未带来额外收益**, 反而可能因模型开始接触数据噪声导致泛化能力轻微下降。
 2. **早停机制有效**, 避免了更差的结果, 但最佳验证点仍低于阶段十。
 3. 训练后期验证准确率在 91.5% 附近波动, 未能突破阶段十的 92.38%, 表明模型可能已接近当前配置下的性能上限。
- **训练时间过长:** 80 epoch 对于该模型和数据可能过多, 后期学习率极低, 参数更新过于精细, 容易拟合训练数据中的噪声。
 - **数据增强的随机性:** RandAugment 和 MixUp 的组合可能导致某些 epoch 的验证准确率波动较大, 最佳模型出现时间较晚但未能超过之前。

- **模型容量限制：**SEResNet 参数量 1127 万，对于该数据集可能已接近容量极限，进一步提升需要更先进的架构或预训练。

阶段十二：最终集成学习模型

Plain Text

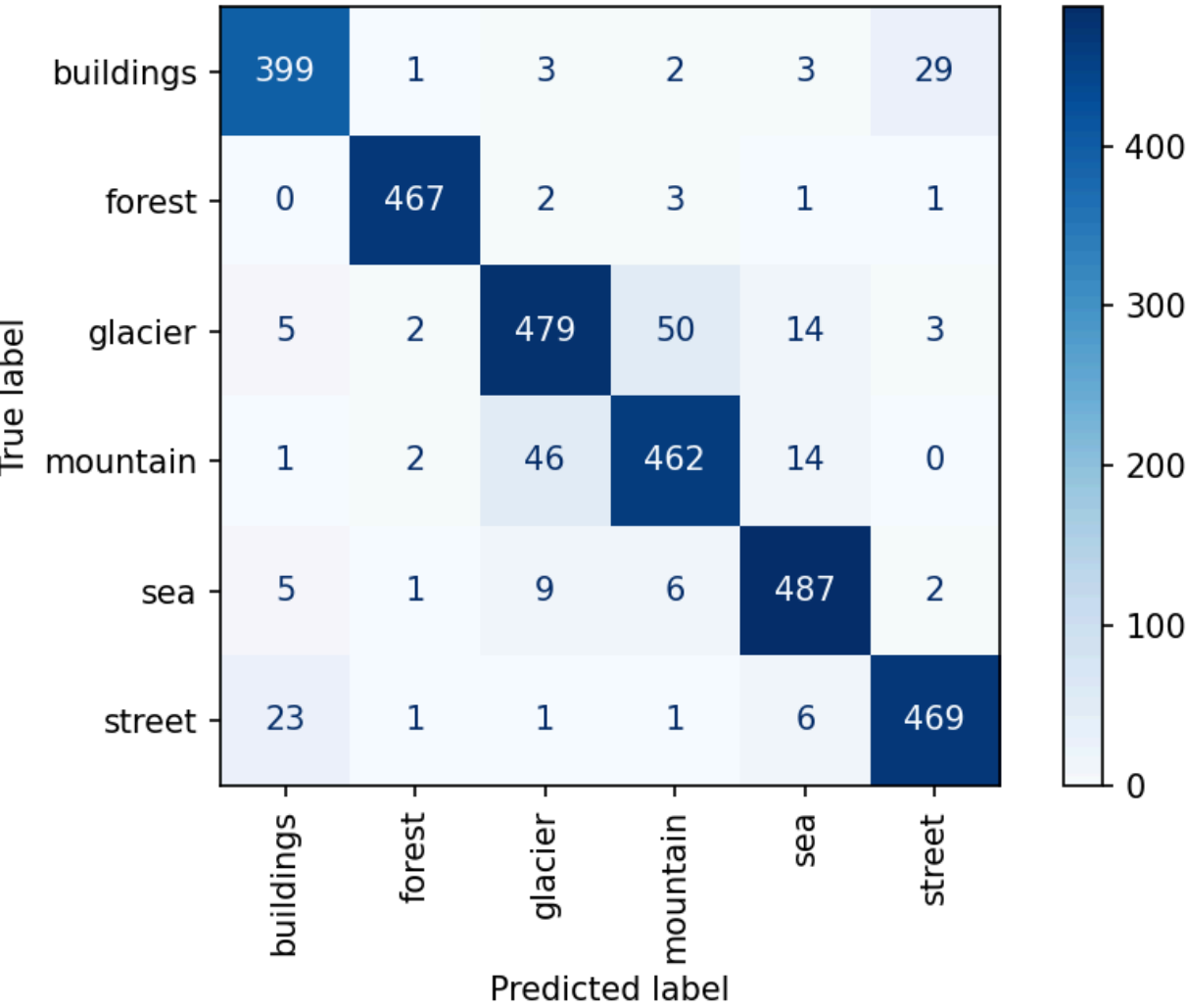
```
1 ensemble_final.py
```

结果

Plain Text

```
1 集成模型测试准确率：0.9210
2
3 分类报告：
4           precision    recall  f1-score   support
5
6  buildings      0.9215      0.9130      0.9172        437
7    forest      0.9852      0.9852      0.9852        474
8   glacier      0.8870      0.8662      0.8765        553
9   mountain      0.8817      0.8800      0.8808        525
10         sea      0.9276      0.9549      0.9411        510
11        street      0.9306      0.9361      0.9333        501
12
13    accuracy                    0.9210        3000
14   macro avg      0.9223      0.9226      0.9224        3000
15 weighted avg      0.9208      0.9210      0.9208        3000
```

Ensemble (Stage7+8+10+11) Confusion Matrix (Acc: 92.10%)

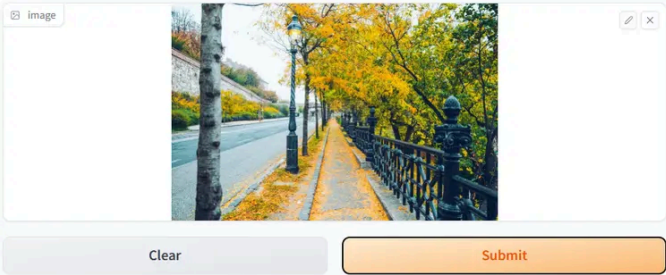


横向对比结果

- 加载阶段 1、4、7、12 的实验结果，生成以下对比图：
 - 各阶段训练 / 验证准确率曲线对比（并列或叠加）
 - 各阶段训练 / 验证损失曲线对比
 - 各阶段测试准确率柱状图
 - 各阶段各类别召回率对比柱状图
 - 各阶段混淆矩阵并列展示
- 加载阶段 1、4、7、12 的模型，提供上传图片功能，显示四个模型的预测结果，并生成阶段 1、4、7 和阶段 12 的 Grad-CAM 热力图对比

Python

```
1 compare_stages_final.py      # 对比图
2 gradio_demo_final.py        # Demo
```



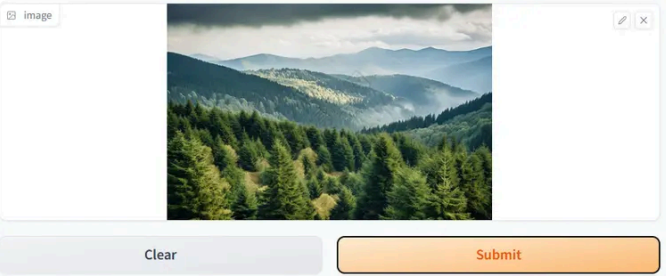
output 0

Stage	Predicted Class	Confidence
Stage1	street	0.5295
Stage4	street	0.5717
Stage7	street	0.8089
Stage12 (Ensemble)	street	0.8605

Grad-CAM (Stage1,4,7,12) 1 (pred: street)

Stage7 (pred: street)

Stage12 (using Stage10) (pred: street)



output 0

Stage	Predicted Class	Confidence
Stage1	mountain	0.8915
Stage4	mountain	0.9473
Stage7	mountain	0.9047
Stage12 (Ensemble)	mountain	0.8934

Grad-CAM (Stage1,4,7,12) 1 (pred: mountain)

Stage7 (pred: mountain)

Stage12 (using Stage10) (pred: mountain)

消融实验

阶段十为基准（最好的单模型）

待消融组件	实验目的
MixUp	验证混合增强的必要性
RandAugment	验证自动增强策略的贡献
SE 注意力	验证通道注意力的有效性
Dropout (0.3)	验证全连接层前随机丢弃的作用
权重衰减 (5e-4)	验证 L2 正则化的强度影响
标签平滑	验证软标签对泛化的帮助
余弦退火 + 预热	验证学习率调度的必要性

Python

```
1 python ablate_no_mixup.py
2 python ablate_no_randaugment.py
3 python ablate_no_se.py
4 python ablate_no_dropout.py
5 python ablate_no_weight_decay.py
6 python ablate_no_label_smoothing.py
7 python ablate_no_cosine.py
8 summarize_ablation.py
9
10 train_stage10_no_mixup_no_ls_no_wd.py
```

ablate_no_mixup

Python

```
1 使用设备: cuda
2 模型参数量: 11273470
3 Epoch 1/40: Train Loss: 1.4386, Acc: 0.4605 | Val Loss: 1.0988, Acc: 0.6669 | LR: 2.08e-04
4 Epoch 2/40: Train Loss: 1.0927, Acc: 0.6635 | Val Loss: 0.9139, Acc: 0.7691 | LR: 4.06e-04
5 Epoch 3/40: Train Loss: 0.9766, Acc: 0.7250 | Val Loss: 0.8712, Acc: 0.7830 | LR: 6.04e-04
6 Epoch 4/40: Train Loss: 0.9061, Acc: 0.7634 | Val Loss: 0.8456, Acc: 0.7987 | LR: 8.02e-04
7 Epoch 5/40: Train Loss: 0.8875, Acc: 0.7745 | Val Loss: 0.7830, Acc: 0.8336 | LR: 1.00e-03
8 Epoch 6/40: Train Loss: 0.8557, Acc: 0.7913 | Val Loss: 0.7991, Acc: 0.8155 | LR: 9.98e-04
9 Epoch 7/40: Train Loss: 0.8179, Acc: 0.8095 | Val Loss: 0.8053, Acc: 0.8108 | LR: 9.92e-04
10 Epoch 8/40: Train Loss: 0.7997, Acc: 0.8223 | Val Loss: 0.7095, Acc: 0.8675 | LR: 9.82e-04
11 Epoch 9/40: Train Loss: 0.7850, Acc: 0.8293 | Val Loss: 0.9212, Acc: 0.7748 | LR: 9.68e-04
12 Epoch 10/40: Train Loss: 0.7656, Acc: 0.8392 | Val Loss: 0.7183, Acc: 0.8561 | LR: 9.51e-04
13 Epoch 11/40: Train Loss: 0.7505, Acc: 0.8407 | Val Loss: 0.6689, Acc: 0.8821 | LR: 9.29e-04
14 Epoch 12/40: Train Loss: 0.7339, Acc: 0.8547 | Val Loss: 0.6489, Acc: 0.8892 | LR: 9.05e-04
15 Epoch 13/40: Train Loss: 0.7297, Acc: 0.8519 | Val Loss: 0.6398, Acc: 0.8970 | LR: 8.77e-04
16 Epoch 14/40: Train Loss: 0.7152, Acc: 0.8589 | Val Loss: 0.7348, Acc: 0.8493 | LR: 8.46e-04
17 Epoch 15/40: Train Loss: 0.7083, Acc: 0.8635 | Val Loss: 0.7112, Acc: 0.8650 | LR: 8.12e-04
18 Epoch 16/40: Train Loss: 0.6933, Acc: 0.8719 | Val Loss: 0.6468, Acc: 0.8942 | LR: 7.76e-04
19 Epoch 17/40: Train Loss: 0.6842, Acc: 0.8746 | Val Loss: 0.6453, Acc: 0.8970 | LR: 7.37e-04
20 Epoch 18/40: Train Loss: 0.6779, Acc: 0.8784 | Val Loss: 0.6127, Acc: 0.9081 | LR: 6.97e-04
21 Epoch 19/40: Train Loss: 0.6723, Acc: 0.8805 | Val Loss: 0.6536, Acc:
```


0.8956 | LR: 6.55e-04

22 Epoch 20/40: Train Loss: 0.6559, Acc: 0.8876 | Val Loss: 0.6970, Acc: 0.8760 | LR: 6.12e-04

23 Epoch 21/40: Train Loss: 0.6505, Acc: 0.8926 | Val Loss: 0.6183, Acc: 0.9095 | LR: 5.68e-04

24 Epoch 22/40: Train Loss: 0.6485, Acc: 0.8946 | Val Loss: 0.6761, Acc: 0.8799 | LR: 5.23e-04

25 Epoch 23/40: Train Loss: 0.6293, Acc: 0.9005 | Val Loss: 0.6226, Acc: 0.9038 | LR: 4.78e-04

26 Epoch 24/40: Train Loss: 0.6233, Acc: 0.9033 | Val Loss: 0.5993, Acc: 0.9141 | LR: 4.33e-04

27 Epoch 25/40: Train Loss: 0.6189, Acc: 0.9067 | Val Loss: 0.6467, Acc: 0.8906 | LR: 3.89e-04

28 Epoch 26/40: Train Loss: 0.6032, Acc: 0.9148 | Val Loss: 0.6198, Acc: 0.9024 | LR: 3.46e-04

29 Epoch 27/40: Train Loss: 0.6025, Acc: 0.9169 | Val Loss: 0.6023, Acc: 0.9195 | LR: 3.04e-04

30 Epoch 28/40: Train Loss: 0.5886, Acc: 0.9207 | Val Loss: 0.5896, Acc: 0.9227 | LR: 2.64e-04

31 Epoch 29/40: Train Loss: 0.5873, Acc: 0.9221 | Val Loss: 0.5822, Acc: 0.9245 | LR: 2.25e-04

32 Epoch 30/40: Train Loss: 0.5726, Acc: 0.9281 | Val Loss: 0.5796, Acc: 0.9259 | LR: 1.89e-04

33 Epoch 31/40: Train Loss: 0.5679, Acc: 0.9322 | Val Loss: 0.5715, Acc: 0.9316 | LR: 1.55e-04

34 Epoch 32/40: Train Loss: 0.5530, Acc: 0.9378 | Val Loss: 0.5738, Acc: 0.9284 | LR: 1.24e-04

35 Epoch 33/40: Train Loss: 0.5499, Acc: 0.9405 | Val Loss: 0.5739, Acc: 0.9277 | LR: 9.64e-05

36 Epoch 34/40: Train Loss: 0.5404, Acc: 0.9449 | Val Loss: 0.5738, Acc: 0.9305 | LR: 7.17e-05

37 Epoch 35/40: Train Loss: 0.5351, Acc: 0.9494 | Val Loss: 0.5706, Acc: 0.9316 | LR: 5.05e-05

38 Epoch 36/40: Train Loss: 0.5321, Acc: 0.9500 | Val Loss: 0.5703, Acc: 0.9305 | LR: 3.29e-05

39 Epoch 37/40: Train Loss: 0.5316, Acc: 0.9519 | Val Loss: 0.5692, Acc: 0.9344 | LR: 1.90e-05

40 Epoch 38/40: Train Loss: 0.5257, Acc: 0.9546 | Val Loss: 0.5701, Acc: 0.9320 | LR: 9.03e-06

41 Epoch 39/40: Train Loss: 0.5251, Acc: 0.9533 | Val Loss: 0.5733, Acc: 0.9309 | LR: 3.01e-06

42 Epoch 40/40: Train Loss: 0.5186, Acc: 0.9590 | Val Loss: 0.5703, Acc:
0.9312 | LR: 1.00e-06
43 测试准确率: 0.9153

ablate_no_randaugment

Python

```
1 使用设备: cuda
2 类别名称: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
3 模型参数量: 11273470
4 开始训练 (去掉 RandAugment) ...
5 Epoch 1/40: Train Loss: 1.3113, Acc: 0.5377 | Val Loss: 1.0474, Acc: 0.7047 | LR: 2.08e-04
6 Epoch 2/40: Train Loss: 0.9816, Acc: 0.7304 | Val Loss: 0.8413, Acc: 0.7952 | LR: 4.06e-04
7 Epoch 3/40: Train Loss: 0.8758, Acc: 0.7830 | Val Loss: 0.9563, Acc: 0.7364 | LR: 6.04e-04
8 Epoch 4/40: Train Loss: 0.8110, Acc: 0.8189 | Val Loss: 1.0171, Acc: 0.7189 | LR: 8.02e-04
9 Epoch 5/40: Train Loss: 0.7921, Acc: 0.8263 | Val Loss: 0.8125, Acc: 0.8062 | LR: 1.00e-03
10 Epoch 6/40: Train Loss: 0.7624, Acc: 0.8424 | Val Loss: 0.7771, Acc: 0.8279 | LR: 9.98e-04
11 Epoch 7/40: Train Loss: 0.7364, Acc: 0.8561 | Val Loss: 0.7645, Acc: 0.8429 | LR: 9.92e-04
12 Epoch 8/40: Train Loss: 0.7029, Acc: 0.8708 | Val Loss: 0.7364, Acc: 0.8504 | LR: 9.82e-04
13 Epoch 9/40: Train Loss: 0.6876, Acc: 0.8762 | Val Loss: 0.7031, Acc: 0.8664 | LR: 9.68e-04
14 Epoch 10/40: Train Loss: 0.6752, Acc: 0.8835 | Val Loss: 0.7635, Acc: 0.8358 | LR: 9.51e-04
15 Epoch 11/40: Train Loss: 0.6608, Acc: 0.8896 | Val Loss: 0.8034, Acc: 0.8151 | LR: 9.29e-04
16 Epoch 12/40: Train Loss: 0.6512, Acc: 0.8947 | Val Loss: 0.7548, Acc: 0.8425 | LR: 9.05e-04
17 Epoch 13/40: Train Loss: 0.6344, Acc: 0.9039 | Val Loss: 0.7091, Acc: 0.8614 | LR: 8.77e-04
18 Epoch 14/40: Train Loss: 0.6191, Acc: 0.9065 | Val Loss: 0.6731, Acc: 0.8821 | LR: 8.46e-04
19 Epoch 15/40: Train Loss: 0.6078, Acc: 0.9139 | Val Loss: 0.7041, Acc: 0.8625 | LR: 8.12e-04
20 Epoch 16/40: Train Loss: 0.5983, Acc: 0.9211 | Val Loss: 0.7317, Acc: 0.8643 | LR: 7.76e-04
21 Epoch 17/40: Train Loss: 0.5858, Acc: 0.9257 | Val Loss: 0.6725, Acc: 0.8839 | LR: 7.37e-04
22 Epoch 18/40: Train Loss: 0.5676, Acc: 0.9336 | Val Loss: 0.6513, Acc:
```

0.8921 | LR: 6.97e-04

23 Epoch 19/40: Train Loss: 0.5666, Acc: 0.9336 | Val Loss: 0.7061, Acc: 0.8614 | LR: 6.55e-04

24 Epoch 20/40: Train Loss: 0.5560, Acc: 0.9390 | Val Loss: 0.6478, Acc: 0.9006 | LR: 6.12e-04

25 Epoch 21/40: Train Loss: 0.5383, Acc: 0.9485 | Val Loss: 0.6932, Acc: 0.8778 | LR: 5.68e-04

26 Epoch 22/40: Train Loss: 0.5303, Acc: 0.9523 | Val Loss: 0.6723, Acc: 0.8888 | LR: 5.23e-04

27 Epoch 23/40: Train Loss: 0.5181, Acc: 0.9584 | Val Loss: 0.6888, Acc: 0.8856 | LR: 4.78e-04

28 Epoch 24/40: Train Loss: 0.5061, Acc: 0.9621 | Val Loss: 0.6643, Acc: 0.8878 | LR: 4.33e-04

29 Epoch 25/40: Train Loss: 0.4884, Acc: 0.9740 | Val Loss: 0.6468, Acc: 0.9002 | LR: 3.89e-04

30 Epoch 26/40: Train Loss: 0.4795, Acc: 0.9769 | Val Loss: 0.6548, Acc: 0.8999 | LR: 3.46e-04

31 Epoch 27/40: Train Loss: 0.4702, Acc: 0.9813 | Val Loss: 0.6932, Acc: 0.8831 | LR: 3.04e-04

32 Epoch 28/40: Train Loss: 0.4682, Acc: 0.9826 | Val Loss: 0.6673, Acc: 0.9017 | LR: 2.64e-04

33 Epoch 29/40: Train Loss: 0.4545, Acc: 0.9884 | Val Loss: 0.6551, Acc: 0.9031 | LR: 2.25e-04

34 Epoch 30/40: Train Loss: 0.4514, Acc: 0.9901 | Val Loss: 0.6533, Acc: 0.9088 | LR: 1.89e-04

35 Epoch 31/40: Train Loss: 0.4510, Acc: 0.9901 | Val Loss: 0.6491, Acc: 0.9077 | LR: 1.55e-04

36 Epoch 32/40: Train Loss: 0.4444, Acc: 0.9927 | Val Loss: 0.6512, Acc: 0.9084 | LR: 1.24e-04

37 Epoch 33/40: Train Loss: 0.4390, Acc: 0.9957 | Val Loss: 0.6530, Acc: 0.9074 | LR: 9.64e-05

38 Epoch 34/40: Train Loss: 0.4375, Acc: 0.9963 | Val Loss: 0.6448, Acc: 0.9141 | LR: 7.17e-05

39 Epoch 35/40: Train Loss: 0.4370, Acc: 0.9963 | Val Loss: 0.6537, Acc: 0.9099 | LR: 5.05e-05

40 Epoch 36/40: Train Loss: 0.4351, Acc: 0.9971 | Val Loss: 0.6492, Acc: 0.9106 | LR: 3.29e-05

41 Epoch 37/40: Train Loss: 0.4342, Acc: 0.9971 | Val Loss: 0.6482, Acc: 0.9113 | LR: 1.90e-05

42 Epoch 38/40: Train Loss: 0.4341, Acc: 0.9970 | Val Loss: 0.6464, Acc: 0.9138 | LR: 9.03e-06

43 Epoch 39/40: Train Loss: 0.4329, Acc: 0.9980 | Val Loss: 0.6446, Acc:
0.9152 | LR: 3.01e-06
44 Epoch 40/40: Train Loss: 0.4323, Acc: 0.9981 | Val Loss: 0.6469, Acc:
0.9116 | LR: 1.00e-06
45 训练完成, 耗时: 2642.82秒
46 在测试集上评估...
47 测试准确率: 0.9053
48 消融实验完成, 结果已保存。

ablate_no_se

Python

```
1 使用设备: cuda
2 类别名称: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
3 模型参数量: 11184390
4 开始训练 (去掉 SE 注意力) ...
5 Epoch 1/40: Train Loss: 1.4950, Acc: 0.3148 | Val Loss: 1.1015, Acc: 0.6580 | LR: 2.08e-04
6 Epoch 2/40: Train Loss: 1.3299, Acc: 0.3993 | Val Loss: 0.9775, Acc: 0.7296 | LR: 4.06e-04
7 Epoch 3/40: Train Loss: 1.1851, Acc: 0.4661 | Val Loss: 0.8369, Acc: 0.8026 | LR: 6.04e-04
8 Epoch 4/40: Train Loss: 1.1141, Acc: 0.4495 | Val Loss: 0.8889, Acc: 0.7781 | LR: 8.02e-04
9 Epoch 5/40: Train Loss: 1.0823, Acc: 0.4596 | Val Loss: 0.9013, Acc: 0.7674 | LR: 1.00e-03
10 Epoch 6/40: Train Loss: 1.0622, Acc: 0.4948 | Val Loss: 0.8849, Acc: 0.7870 | LR: 9.98e-04
11 Epoch 7/40: Train Loss: 1.0297, Acc: 0.4624 | Val Loss: 0.8383, Acc: 0.8073 | LR: 9.92e-04
12 Epoch 8/40: Train Loss: 0.9971, Acc: 0.4389 | Val Loss: 0.9152, Acc: 0.7659 | LR: 9.82e-04
13 Epoch 9/40: Train Loss: 0.9793, Acc: 0.5098 | Val Loss: 0.7903, Acc: 0.8368 | LR: 9.68e-04
14 Epoch 10/40: Train Loss: 0.9834, Acc: 0.5090 | Val Loss: 0.7582, Acc: 0.8454 | LR: 9.51e-04
15 Epoch 11/40: Train Loss: 0.9536, Acc: 0.4863 | Val Loss: 0.7783, Acc: 0.8286 | LR: 9.29e-04
16 Epoch 12/40: Train Loss: 0.9679, Acc: 0.5178 | Val Loss: 0.7610, Acc: 0.8482 | LR: 9.05e-04
17 Epoch 13/40: Train Loss: 0.9446, Acc: 0.5160 | Val Loss: 0.6882, Acc: 0.8760 | LR: 8.77e-04
18 Epoch 14/40: Train Loss: 0.9320, Acc: 0.5057 | Val Loss: 0.6918, Acc: 0.8742 | LR: 8.46e-04
19 Epoch 15/40: Train Loss: 0.9131, Acc: 0.5248 | Val Loss: 0.6902, Acc: 0.8789 | LR: 8.12e-04
20 Epoch 16/40: Train Loss: 0.9203, Acc: 0.5052 | Val Loss: 0.7012, Acc: 0.8785 | LR: 7.76e-04
21 Epoch 17/40: Train Loss: 0.8827, Acc: 0.4936 | Val Loss: 0.7085, Acc: 0.8696 | LR: 7.37e-04
22 Epoch 18/40: Train Loss: 0.8832, Acc: 0.5472 | Val Loss: 0.7281, Acc:
```

0.8539 | LR: 6.97e-04

23 Epoch 19/40: Train Loss: 0.9046, Acc: 0.4805 | Val Loss: 0.7010, Acc: 0.8785 | LR: 6.55e-04

24 Epoch 20/40: Train Loss: 0.8790, Acc: 0.5052 | Val Loss: 0.6953, Acc: 0.8750 | LR: 6.12e-04

25 Epoch 21/40: Train Loss: 0.8770, Acc: 0.5521 | Val Loss: 0.7329, Acc: 0.8543 | LR: 5.68e-04

26 Epoch 22/40: Train Loss: 0.8761, Acc: 0.5585 | Val Loss: 0.6949, Acc: 0.8696 | LR: 5.23e-04

27 Epoch 23/40: Train Loss: 0.8420, Acc: 0.5034 | Val Loss: 0.6909, Acc: 0.8785 | LR: 4.78e-04

28 Epoch 24/40: Train Loss: 0.8568, Acc: 0.5299 | Val Loss: 0.6552, Acc: 0.8938 | LR: 4.33e-04

29 Epoch 25/40: Train Loss: 0.8467, Acc: 0.5116 | Val Loss: 0.6544, Acc: 0.8985 | LR: 3.89e-04

30 Epoch 26/40: Train Loss: 0.8408, Acc: 0.5518 | Val Loss: 0.6438, Acc: 0.8995 | LR: 3.46e-04

31 Epoch 27/40: Train Loss: 0.8187, Acc: 0.4993 | Val Loss: 0.6611, Acc: 0.8910 | LR: 3.04e-04

32 Epoch 28/40: Train Loss: 0.8383, Acc: 0.4905 | Val Loss: 0.6554, Acc: 0.8970 | LR: 2.64e-04

33 Epoch 29/40: Train Loss: 0.8510, Acc: 0.5893 | Val Loss: 0.6385, Acc: 0.9084 | LR: 2.25e-04

34 Epoch 30/40: Train Loss: 0.8046, Acc: 0.5507 | Val Loss: 0.6390, Acc: 0.9049 | LR: 1.89e-04

35 Epoch 31/40: Train Loss: 0.8072, Acc: 0.5527 | Val Loss: 0.6296, Acc: 0.8988 | LR: 1.55e-04

36 Epoch 32/40: Train Loss: 0.8099, Acc: 0.5716 | Val Loss: 0.6418, Acc: 0.9049 | LR: 1.24e-04

37 Epoch 33/40: Train Loss: 0.7827, Acc: 0.4929 | Val Loss: 0.6250, Acc: 0.9067 | LR: 9.64e-05

38 Epoch 34/40: Train Loss: 0.7742, Acc: 0.5625 | Val Loss: 0.6237, Acc: 0.9084 | LR: 7.17e-05

39 Epoch 35/40: Train Loss: 0.7527, Acc: 0.5594 | Val Loss: 0.6225, Acc: 0.9031 | LR: 5.05e-05

40 Epoch 36/40: Train Loss: 0.7361, Acc: 0.5700 | Val Loss: 0.6341, Acc: 0.9067 | LR: 3.29e-05

41 Epoch 37/40: Train Loss: 0.7537, Acc: 0.5991 | Val Loss: 0.6193, Acc: 0.9067 | LR: 1.90e-05

42 Epoch 38/40: Train Loss: 0.7491, Acc: 0.5795 | Val Loss: 0.6206, Acc: 0.9088 | LR: 9.03e-06

```
43 Epoch 39/40: Train Loss: 0.7593, Acc: 0.5463 | Val Loss: 0.6192, Acc:
    0.9067 | LR: 3.01e-06
44 Epoch 40/40: Train Loss: 0.7681, Acc: 0.5494 | Val Loss: 0.6257, Acc:
    0.9067 | LR: 1.00e-06
45 训练完成，耗时：2502.16秒
46 在测试集上评估...
47 测试准确率：0.9140
48 消融实验完成，结果已保存。
```

ablate_no_dropout

Python

```
1 使用设备: cuda
2 类别名称: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
3 模型参数量: 11273470
4 开始训练 (去掉 Dropout) ...
5 Epoch 1/40: Train Loss: 1.4631, Acc: 0.3272 | Val Loss: 1.0901, Acc: 0.6844 | LR: 2.08e-04
6 Epoch 2/40: Train Loss: 1.2086, Acc: 0.4312 | Val Loss: 0.9532, Acc: 0.7403 | LR: 4.06e-04
7 Epoch 3/40: Train Loss: 1.1216, Acc: 0.4472 | Val Loss: 0.8617, Acc: 0.8030 | LR: 6.04e-04
8 Epoch 4/40: Train Loss: 1.0996, Acc: 0.4267 | Val Loss: 0.9199, Acc: 0.7813 | LR: 8.02e-04
9 Epoch 5/40: Train Loss: 1.0372, Acc: 0.4698 | Val Loss: 0.8038, Acc: 0.8301 | LR: 1.00e-03
10 Epoch 6/40: Train Loss: 1.0387, Acc: 0.5232 | Val Loss: 0.9417, Acc: 0.7442 | LR: 9.98e-04
11 Epoch 7/40: Train Loss: 1.0235, Acc: 0.4922 | Val Loss: 0.7958, Acc: 0.8276 | LR: 9.92e-04
12 Epoch 8/40: Train Loss: 0.9834, Acc: 0.4602 | Val Loss: 0.7871, Acc: 0.8286 | LR: 9.82e-04
13 Epoch 9/40: Train Loss: 0.9680, Acc: 0.4963 | Val Loss: 0.7710, Acc: 0.8418 | LR: 9.68e-04
14 Epoch 10/40: Train Loss: 0.9591, Acc: 0.5383 | Val Loss: 0.6911, Acc: 0.8675 | LR: 9.51e-04
15 Epoch 11/40: Train Loss: 0.9323, Acc: 0.5147 | Val Loss: 0.7250, Acc: 0.8625 | LR: 9.29e-04
16 Epoch 12/40: Train Loss: 0.9099, Acc: 0.5245 | Val Loss: 0.7036, Acc: 0.8668 | LR: 9.05e-04
17 Epoch 13/40: Train Loss: 0.9101, Acc: 0.5017 | Val Loss: 0.7254, Acc: 0.8596 | LR: 8.77e-04
18 Epoch 14/40: Train Loss: 0.8984, Acc: 0.5053 | Val Loss: 0.7806, Acc: 0.8518 | LR: 8.46e-04
19 Epoch 15/40: Train Loss: 0.9386, Acc: 0.5415 | Val Loss: 0.7460, Acc: 0.8468 | LR: 8.12e-04
20 Epoch 16/40: Train Loss: 0.8890, Acc: 0.5229 | Val Loss: 0.7411, Acc: 0.8543 | LR: 7.76e-04
21 Epoch 17/40: Train Loss: 0.8769, Acc: 0.5157 | Val Loss: 0.7482, Acc: 0.8457 | LR: 7.37e-04
22 Epoch 18/40: Train Loss: 0.8743, Acc: 0.5785 | Val Loss: 0.7050, Acc:
```

0.8678 | LR: 6.97e-04

23 Epoch 19/40: Train Loss: 0.8882, Acc: 0.5492 | Val Loss: 0.7434, Acc: 0.8436 | LR: 6.55e-04

24 Epoch 20/40: Train Loss: 0.8587, Acc: 0.5127 | Val Loss: 0.7089, Acc: 0.8675 | LR: 6.12e-04

25 Epoch 21/40: Train Loss: 0.8574, Acc: 0.5580 | Val Loss: 0.6936, Acc: 0.8750 | LR: 5.68e-04

26 Epoch 22/40: Train Loss: 0.8336, Acc: 0.5307 | Val Loss: 0.6731, Acc: 0.8867 | LR: 5.23e-04

27 Epoch 23/40: Train Loss: 0.8421, Acc: 0.5325 | Val Loss: 0.6378, Acc: 0.9035 | LR: 4.78e-04

28 Epoch 24/40: Train Loss: 0.8443, Acc: 0.5467 | Val Loss: 0.6514, Acc: 0.8935 | LR: 4.33e-04

29 Epoch 25/40: Train Loss: 0.8033, Acc: 0.5574 | Val Loss: 0.6525, Acc: 0.8935 | LR: 3.89e-04

30 Epoch 26/40: Train Loss: 0.7810, Acc: 0.5082 | Val Loss: 0.6463, Acc: 0.8949 | LR: 3.46e-04

31 Epoch 27/40: Train Loss: 0.8023, Acc: 0.5533 | Val Loss: 0.6707, Acc: 0.8828 | LR: 3.04e-04

32 Epoch 28/40: Train Loss: 0.7746, Acc: 0.5008 | Val Loss: 0.6162, Acc: 0.9084 | LR: 2.64e-04

33 Epoch 29/40: Train Loss: 0.7929, Acc: 0.4964 | Val Loss: 0.6412, Acc: 0.8999 | LR: 2.25e-04

34 Epoch 30/40: Train Loss: 0.7744, Acc: 0.5159 | Val Loss: 0.6366, Acc: 0.9035 | LR: 1.89e-04

35 Epoch 31/40: Train Loss: 0.7748, Acc: 0.5836 | Val Loss: 0.6339, Acc: 0.8999 | LR: 1.55e-04

36 Epoch 32/40: Train Loss: 0.7404, Acc: 0.6042 | Val Loss: 0.6174, Acc: 0.9088 | LR: 1.24e-04

37 Epoch 33/40: Train Loss: 0.7246, Acc: 0.5588 | Val Loss: 0.6274, Acc: 0.9106 | LR: 9.64e-05

38 Epoch 34/40: Train Loss: 0.7689, Acc: 0.5834 | Val Loss: 0.6341, Acc: 0.9031 | LR: 7.17e-05

39 Epoch 35/40: Train Loss: 0.7211, Acc: 0.5950 | Val Loss: 0.6167, Acc: 0.9102 | LR: 5.05e-05

40 Epoch 36/40: Train Loss: 0.7632, Acc: 0.5961 | Val Loss: 0.6246, Acc: 0.9095 | LR: 3.29e-05

41 Epoch 37/40: Train Loss: 0.7169, Acc: 0.5587 | Val Loss: 0.6207, Acc: 0.9116 | LR: 1.90e-05

42 Epoch 38/40: Train Loss: 0.7286, Acc: 0.5941 | Val Loss: 0.6164, Acc: 0.9131 | LR: 9.03e-06

43 Epoch 39/40: Train Loss: 0.7465, Acc: 0.5081 | Val Loss: 0.6123, Acc: 0.9102 | LR: 3.01e-06
44 Epoch 40/40: Train Loss: 0.7086, Acc: 0.5488 | Val Loss: 0.6165, Acc: 0.9102 | LR: 1.00e-06
45 训练完成，耗时：2587.40秒
46 在测试集上评估...
47 测试准确率：0.9140
48 消融实验完成，结果已保存。

ablate_no_weight_decay

Python

1 # 没必要可能，晚点试

ablate_no_label_smoothing

Python

```
1 使用设备: cuda
2 类别名称: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
3 模型参数量: 11273470
4 开始训练 (去掉标签平滑) ...
5 Epoch 1/40: Train Loss: 1.4395, Acc: 0.3033 | Val Loss: 0.9308, Acc: 0.6751 | LR: 2.08e-04
6 Epoch 2/40: Train Loss: 1.0986, Acc: 0.4332 | Val Loss: 0.8122, Acc: 0.6861 | LR: 4.06e-04
7 Epoch 3/40: Train Loss: 0.9570, Acc: 0.4352 | Val Loss: 0.7582, Acc: 0.7064 | LR: 6.04e-04
8 Epoch 4/40: Train Loss: 0.9079, Acc: 0.4836 | Val Loss: 0.5942, Acc: 0.7795 | LR: 8.02e-04
9 Epoch 5/40: Train Loss: 0.8667, Acc: 0.4583 | Val Loss: 0.9321, Acc: 0.6437 | LR: 1.00e-03
10 Epoch 6/40: Train Loss: 0.8488, Acc: 0.4927 | Val Loss: 0.4711, Acc: 0.8422 | LR: 9.98e-04
11 Epoch 7/40: Train Loss: 0.8072, Acc: 0.4954 | Val Loss: 0.5252, Acc: 0.8333 | LR: 9.92e-04
12 Epoch 8/40: Train Loss: 0.7832, Acc: 0.5333 | Val Loss: 0.4935, Acc: 0.8290 | LR: 9.82e-04
13 Epoch 9/40: Train Loss: 0.7535, Acc: 0.5153 | Val Loss: 0.4878, Acc: 0.8187 | LR: 9.68e-04
14 Epoch 10/40: Train Loss: 0.7589, Acc: 0.5394 | Val Loss: 0.4681, Acc: 0.8340 | LR: 9.51e-04
15 Epoch 11/40: Train Loss: 0.7326, Acc: 0.4984 | Val Loss: 0.4655, Acc: 0.8539 | LR: 9.29e-04
16 Epoch 12/40: Train Loss: 0.6909, Acc: 0.4945 | Val Loss: 0.4659, Acc: 0.8507 | LR: 9.05e-04
17 Epoch 13/40: Train Loss: 0.6878, Acc: 0.4859 | Val Loss: 0.4757, Acc: 0.8361 | LR: 8.77e-04
18 Epoch 14/40: Train Loss: 0.6774, Acc: 0.5102 | Val Loss: 0.4366, Acc: 0.8529 | LR: 8.46e-04
19 Epoch 15/40: Train Loss: 0.6343, Acc: 0.5500 | Val Loss: 0.4156, Acc: 0.8614 | LR: 8.12e-04
20 Epoch 16/40: Train Loss: 0.6900, Acc: 0.5455 | Val Loss: 0.4191, Acc: 0.8543 | LR: 7.76e-04
21 Epoch 17/40: Train Loss: 0.6547, Acc: 0.5565 | Val Loss: 0.3766, Acc: 0.8714 | LR: 7.37e-04
22 Epoch 18/40: Train Loss: 0.6144, Acc: 0.5317 | Val Loss: 0.3784, Acc:
```

0.8760 | LR: 6.97e-04

23 Epoch 19/40: Train Loss: 0.6009, Acc: 0.5343 | Val Loss: 0.3493, Acc: 0.8842 | LR: 6.55e-04

24 Epoch 20/40: Train Loss: 0.6031, Acc: 0.5098 | Val Loss: 0.3590, Acc: 0.8796 | LR: 6.12e-04

25 Epoch 21/40: Train Loss: 0.6155, Acc: 0.4910 | Val Loss: 0.3531, Acc: 0.8817 | LR: 5.68e-04

26 Epoch 22/40: Train Loss: 0.5659, Acc: 0.5295 | Val Loss: 0.3190, Acc: 0.8917 | LR: 5.23e-04

27 Epoch 23/40: Train Loss: 0.5708, Acc: 0.5518 | Val Loss: 0.3363, Acc: 0.8849 | LR: 4.78e-04

28 Epoch 24/40: Train Loss: 0.5716, Acc: 0.5423 | Val Loss: 0.2928, Acc: 0.9027 | LR: 4.33e-04

29 Epoch 25/40: Train Loss: 0.5480, Acc: 0.5676 | Val Loss: 0.3043, Acc: 0.8970 | LR: 3.89e-04

30 Epoch 26/40: Train Loss: 0.5415, Acc: 0.5978 | Val Loss: 0.3029, Acc: 0.9120 | LR: 3.46e-04

31 Epoch 27/40: Train Loss: 0.4984, Acc: 0.5646 | Val Loss: 0.2927, Acc: 0.9045 | LR: 3.04e-04

32 Epoch 28/40: Train Loss: 0.5440, Acc: 0.5580 | Val Loss: 0.2947, Acc: 0.9027 | LR: 2.64e-04

33 Epoch 29/40: Train Loss: 0.4877, Acc: 0.5527 | Val Loss: 0.2843, Acc: 0.9035 | LR: 2.25e-04

34 Epoch 30/40: Train Loss: 0.4936, Acc: 0.5451 | Val Loss: 0.2855, Acc: 0.9045 | LR: 1.89e-04

35 Epoch 31/40: Train Loss: 0.5237, Acc: 0.5767 | Val Loss: 0.2626, Acc: 0.9134 | LR: 1.55e-04

36 Epoch 32/40: Train Loss: 0.4999, Acc: 0.5788 | Val Loss: 0.2708, Acc: 0.9074 | LR: 1.24e-04

37 Epoch 33/40: Train Loss: 0.4205, Acc: 0.5122 | Val Loss: 0.2621, Acc: 0.9099 | LR: 9.64e-05

38 Epoch 34/40: Train Loss: 0.4632, Acc: 0.5712 | Val Loss: 0.2653, Acc: 0.9088 | LR: 7.17e-05

39 Epoch 35/40: Train Loss: 0.4644, Acc: 0.5458 | Val Loss: 0.2696, Acc: 0.9113 | LR: 5.05e-05

40 Epoch 36/40: Train Loss: 0.5129, Acc: 0.5697 | Val Loss: 0.2740, Acc: 0.9127 | LR: 3.29e-05

41 Epoch 37/40: Train Loss: 0.4584, Acc: 0.5632 | Val Loss: 0.2623, Acc: 0.9109 | LR: 1.90e-05

42 Epoch 38/40: Train Loss: 0.4951, Acc: 0.5415 | Val Loss: 0.2616, Acc: 0.9102 | LR: 9.03e-06

43 Epoch 39/40: Train Loss: 0.4369, Acc: 0.5830 | Val Loss: 0.2683, Acc:
0.9102 | LR: 3.01e-06
44 Epoch 40/40: Train Loss: 0.4441, Acc: 0.5354 | Val Loss: 0.2612, Acc:
0.9088 | LR: 1.00e-06
45 训练完成, 耗时: 2622.55秒
46 在测试集上评估...
47 测试准确率: 0.9160
48 消融实验完成, 结果已保存。

ablate_no_cosine

Python

```
1 使用设备: cuda
2 类别名称: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
3 模型参数量: 11273470
4 开始训练 (去掉余弦退火) ...
5 Epoch 1/40: Train Loss: 1.4996, Acc: 0.3184 | Val Loss: 1.1399, Acc: 0.6437 | LR: 2.08e-04
6 Epoch 2/40: Train Loss: 1.2610, Acc: 0.4116 | Val Loss: 0.9461, Acc: 0.7446 | LR: 4.06e-04
7 Epoch 3/40: Train Loss: 1.1286, Acc: 0.4486 | Val Loss: 0.9688, Acc: 0.7303 | LR: 6.04e-04
8 Epoch 4/40: Train Loss: 1.1095, Acc: 0.4194 | Val Loss: 0.8461, Acc: 0.7955 | LR: 8.02e-04
9 Epoch 5/40: Train Loss: 1.1064, Acc: 0.4596 | Val Loss: 0.8686, Acc: 0.7919 | LR: 1.00e-03
10 Epoch 6/40: Train Loss: 1.0481, Acc: 0.5212 | Val Loss: 0.9410, Acc: 0.7275 | LR: 1.00e-03
11 Epoch 7/40: Train Loss: 0.9992, Acc: 0.4717 | Val Loss: 0.7734, Acc: 0.8415 | LR: 1.00e-03
12 Epoch 8/40: Train Loss: 1.0168, Acc: 0.4985 | Val Loss: 0.7653, Acc: 0.8404 | LR: 1.00e-03
13 Epoch 9/40: Train Loss: 1.0226, Acc: 0.5274 | Val Loss: 0.7628, Acc: 0.8454 | LR: 1.00e-03
14 Epoch 10/40: Train Loss: 0.9524, Acc: 0.4897 | Val Loss: 0.7677, Acc: 0.8358 | LR: 1.00e-03
15 Epoch 11/40: Train Loss: 0.9403, Acc: 0.5191 | Val Loss: 0.7237, Acc: 0.8564 | LR: 1.00e-03
16 Epoch 12/40: Train Loss: 0.9732, Acc: 0.4991 | Val Loss: 0.7377, Acc: 0.8518 | LR: 1.00e-03
17 Epoch 13/40: Train Loss: 0.9264, Acc: 0.5120 | Val Loss: 0.7223, Acc: 0.8596 | LR: 1.00e-03
18 Epoch 14/40: Train Loss: 0.9361, Acc: 0.5178 | Val Loss: 0.7302, Acc: 0.8628 | LR: 1.00e-03
19 Epoch 15/40: Train Loss: 0.9325, Acc: 0.5116 | Val Loss: 0.6990, Acc: 0.8657 | LR: 1.00e-03
20 Epoch 16/40: Train Loss: 0.9235, Acc: 0.5120 | Val Loss: 0.7624, Acc: 0.8354 | LR: 1.00e-03
21 Epoch 17/40: Train Loss: 0.8789, Acc: 0.5385 | Val Loss: 0.6981, Acc: 0.8707 | LR: 1.00e-03
22 Epoch 18/40: Train Loss: 0.8534, Acc: 0.5713 | Val Loss: 0.6862, Acc:
```

0.8693 | LR: 1.00e-03

23 Epoch 19/40: Train Loss: 0.9021, Acc: 0.5791 | Val Loss: 0.6805, Acc: 0.8810 | LR: 1.00e-03

24 Epoch 20/40: Train Loss: 0.9002, Acc: 0.5751 | Val Loss: 0.6714, Acc: 0.8839 | LR: 1.00e-03

25 Epoch 21/40: Train Loss: 0.8542, Acc: 0.5437 | Val Loss: 0.6915, Acc: 0.8714 | LR: 1.00e-03

26 Epoch 22/40: Train Loss: 0.8788, Acc: 0.5004 | Val Loss: 0.6813, Acc: 0.8792 | LR: 1.00e-03

27 Epoch 23/40: Train Loss: 0.8826, Acc: 0.5375 | Val Loss: 0.7228, Acc: 0.8486 | LR: 1.00e-03

28 Epoch 24/40: Train Loss: 0.8776, Acc: 0.5161 | Val Loss: 0.6785, Acc: 0.8824 | LR: 1.00e-03

29 Epoch 25/40: Train Loss: 0.8510, Acc: 0.5142 | Val Loss: 0.6890, Acc: 0.8732 | LR: 1.00e-03

30 Epoch 26/40: Train Loss: 0.8326, Acc: 0.5675 | Val Loss: 0.6543, Acc: 0.8956 | LR: 1.00e-03

31 Epoch 27/40: Train Loss: 0.8220, Acc: 0.5495 | Val Loss: 0.6520, Acc: 0.8888 | LR: 1.00e-03

32 Epoch 28/40: Train Loss: 0.8346, Acc: 0.5299 | Val Loss: 0.7023, Acc: 0.8803 | LR: 1.00e-03

33 Epoch 29/40: Train Loss: 0.8236, Acc: 0.5225 | Val Loss: 0.7095, Acc: 0.8614 | LR: 1.00e-03

34 Epoch 30/40: Train Loss: 0.8213, Acc: 0.5080 | Val Loss: 0.6506, Acc: 0.8956 | LR: 1.00e-03

35 Epoch 31/40: Train Loss: 0.8345, Acc: 0.5973 | Val Loss: 0.6474, Acc: 0.8956 | LR: 1.00e-03

36 Epoch 32/40: Train Loss: 0.8169, Acc: 0.4985 | Val Loss: 0.6793, Acc: 0.8839 | LR: 1.00e-03

37 Epoch 33/40: Train Loss: 0.8316, Acc: 0.5562 | Val Loss: 0.6920, Acc: 0.8700 | LR: 1.00e-03

38 Epoch 34/40: Train Loss: 0.8280, Acc: 0.5542 | Val Loss: 0.6496, Acc: 0.8864 | LR: 1.00e-03

39 Epoch 35/40: Train Loss: 0.8307, Acc: 0.5263 | Val Loss: 0.6356, Acc: 0.8978 | LR: 1.00e-03

40 Epoch 36/40: Train Loss: 0.8169, Acc: 0.5336 | Val Loss: 0.6518, Acc: 0.8913 | LR: 1.00e-03

41 Epoch 37/40: Train Loss: 0.7946, Acc: 0.5347 | Val Loss: 0.6451, Acc: 0.8999 | LR: 1.00e-03

42 Epoch 38/40: Train Loss: 0.8101, Acc: 0.5532 | Val Loss: 0.6605, Acc: 0.8906 | LR: 1.00e-03


```
43 Epoch 39/40: Train Loss: 0.7904, Acc: 0.5863 | Val Loss: 0.6901, Acc:
    0.8732 | LR: 1.00e-03
44 Epoch 40/40: Train Loss: 0.7997, Acc: 0.5258 | Val Loss: 0.6294, Acc:
    0.9045 | LR: 1.00e-03
45 训练完成，耗时：2943.26秒
46 在测试集上评估...
47 测试准确率：0.9073
48 消融实验完成，结果已保存。
```

train_stage10_no_mixup_no_ls_no_wd

去掉前面消融后没有变化的

Python

```
1 使用设备: cuda
2 类别名称: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
3 模型参数量: 11273470
4 开始训练 (去掉 MixUp、标签平滑、权重衰减) ...
5 Epoch 1/40: Train Loss: 1.3562, Acc: 0.4540 | Val Loss: 0.9719, Acc: 0.6177 | LR: 2.08e-04
6 Epoch 2/40: Train Loss: 0.8880, Acc: 0.6594 | Val Loss: 0.6611, Acc: 0.7317 | LR: 4.06e-04
7 Epoch 3/40: Train Loss: 0.7162, Acc: 0.7355 | Val Loss: 0.6106, Acc: 0.7820 | LR: 6.04e-04
8 Epoch 4/40: Train Loss: 0.6467, Acc: 0.7577 | Val Loss: 0.6527, Acc: 0.7638 | LR: 8.02e-04
9 Epoch 5/40: Train Loss: 0.6127, Acc: 0.7712 | Val Loss: 0.5100, Acc: 0.8158 | LR: 1.00e-03
10 Epoch 6/40: Train Loss: 0.5861, Acc: 0.7823 | Val Loss: 0.6151, Acc: 0.7870 | LR: 9.98e-04
11 Epoch 7/40: Train Loss: 0.5306, Acc: 0.8083 | Val Loss: 0.6580, Acc: 0.7788 | LR: 9.92e-04
12 Epoch 8/40: Train Loss: 0.5005, Acc: 0.8166 | Val Loss: 0.5161, Acc: 0.8115 | LR: 9.82e-04
13 Epoch 9/40: Train Loss: 0.4792, Acc: 0.8269 | Val Loss: 0.3967, Acc: 0.8625 | LR: 9.68e-04
14 Epoch 10/40: Train Loss: 0.4575, Acc: 0.8321 | Val Loss: 0.3915, Acc: 0.8600 | LR: 9.51e-04
15 Epoch 11/40: Train Loss: 0.4326, Acc: 0.8421 | Val Loss: 0.3765, Acc: 0.8742 | LR: 9.29e-04
16 Epoch 12/40: Train Loss: 0.4147, Acc: 0.8495 | Val Loss: 0.4387, Acc: 0.8400 | LR: 9.05e-04
17 Epoch 13/40: Train Loss: 0.3956, Acc: 0.8585 | Val Loss: 0.3936, Acc: 0.8621 | LR: 8.77e-04
18 Epoch 14/40: Train Loss: 0.3772, Acc: 0.8624 | Val Loss: 0.3964, Acc: 0.8582 | LR: 8.46e-04
19 Epoch 15/40: Train Loss: 0.3667, Acc: 0.8654 | Val Loss: 0.5674, Acc: 0.8183 | LR: 8.12e-04
20 Epoch 16/40: Train Loss: 0.3464, Acc: 0.8768 | Val Loss: 0.3440, Acc: 0.8732 | LR: 7.76e-04
21 Epoch 17/40: Train Loss: 0.3321, Acc: 0.8796 | Val Loss: 0.3551, Acc: 0.8789 | LR: 7.37e-04
22 Epoch 18/40: Train Loss: 0.3309, Acc: 0.8786 | Val Loss: 0.3194, Acc:
```

0.8949 | LR: 6.97e-04

23 Epoch 19/40: Train Loss: 0.3166, Acc: 0.8871 | Val Loss: 0.3489, Acc: 0.8799 | LR: 6.55e-04

24 Epoch 20/40: Train Loss: 0.3064, Acc: 0.8920 | Val Loss: 0.3625, Acc: 0.8764 | LR: 6.12e-04

25 Epoch 21/40: Train Loss: 0.2927, Acc: 0.8938 | Val Loss: 0.3230, Acc: 0.8867 | LR: 5.68e-04

26 Epoch 22/40: Train Loss: 0.2825, Acc: 0.8961 | Val Loss: 0.3181, Acc: 0.8896 | LR: 5.23e-04

27 Epoch 23/40: Train Loss: 0.2766, Acc: 0.9004 | Val Loss: 0.2999, Acc: 0.9013 | LR: 4.78e-04

28 Epoch 24/40: Train Loss: 0.2622, Acc: 0.9053 | Val Loss: 0.2750, Acc: 0.9049 | LR: 4.33e-04

29 Epoch 25/40: Train Loss: 0.2485, Acc: 0.9118 | Val Loss: 0.2893, Acc: 0.9056 | LR: 3.89e-04

30 Epoch 26/40: Train Loss: 0.2346, Acc: 0.9148 | Val Loss: 0.3743, Acc: 0.8739 | LR: 3.46e-04

31 Epoch 27/40: Train Loss: 0.2234, Acc: 0.9196 | Val Loss: 0.2812, Acc: 0.9074 | LR: 3.04e-04

32 Epoch 28/40: Train Loss: 0.2165, Acc: 0.9188 | Val Loss: 0.2664, Acc: 0.9049 | LR: 2.64e-04

33 Epoch 29/40: Train Loss: 0.2047, Acc: 0.9270 | Val Loss: 0.2728, Acc: 0.9102 | LR: 2.25e-04

34 Epoch 30/40: Train Loss: 0.1889, Acc: 0.9324 | Val Loss: 0.2842, Acc: 0.9084 | LR: 1.89e-04

35 Epoch 31/40: Train Loss: 0.1799, Acc: 0.9363 | Val Loss: 0.2776, Acc: 0.9127 | LR: 1.55e-04

36 Epoch 32/40: Train Loss: 0.1671, Acc: 0.9378 | Val Loss: 0.2809, Acc: 0.9120 | LR: 1.24e-04

37 Epoch 33/40: Train Loss: 0.1620, Acc: 0.9414 | Val Loss: 0.2719, Acc: 0.9141 | LR: 9.64e-05

38 Epoch 34/40: Train Loss: 0.1517, Acc: 0.9451 | Val Loss: 0.2874, Acc: 0.9102 | LR: 7.17e-05

39 Epoch 35/40: Train Loss: 0.1479, Acc: 0.9451 | Val Loss: 0.2764, Acc: 0.9152 | LR: 5.05e-05

40 Epoch 36/40: Train Loss: 0.1403, Acc: 0.9475 | Val Loss: 0.2708, Acc: 0.9149 | LR: 3.29e-05

41 Epoch 37/40: Train Loss: 0.1306, Acc: 0.9523 | Val Loss: 0.2718, Acc: 0.9166 | LR: 1.90e-05

42 Epoch 38/40: Train Loss: 0.1234, Acc: 0.9557 | Val Loss: 0.2767, Acc: 0.9166 | LR: 9.03e-06

43 Epoch 39/40: Train Loss: 0.1261, Acc: 0.9541 | Val Loss: 0.2714, Acc: 0.9170 | LR: 3.01e-06

44 Epoch 40/40: Train Loss: 0.1240, Acc: 0.9557 | Val Loss: 0.2730, Acc: 0.9156 | LR: 1.00e-06

45 训练完成,耗时: 2861.09秒

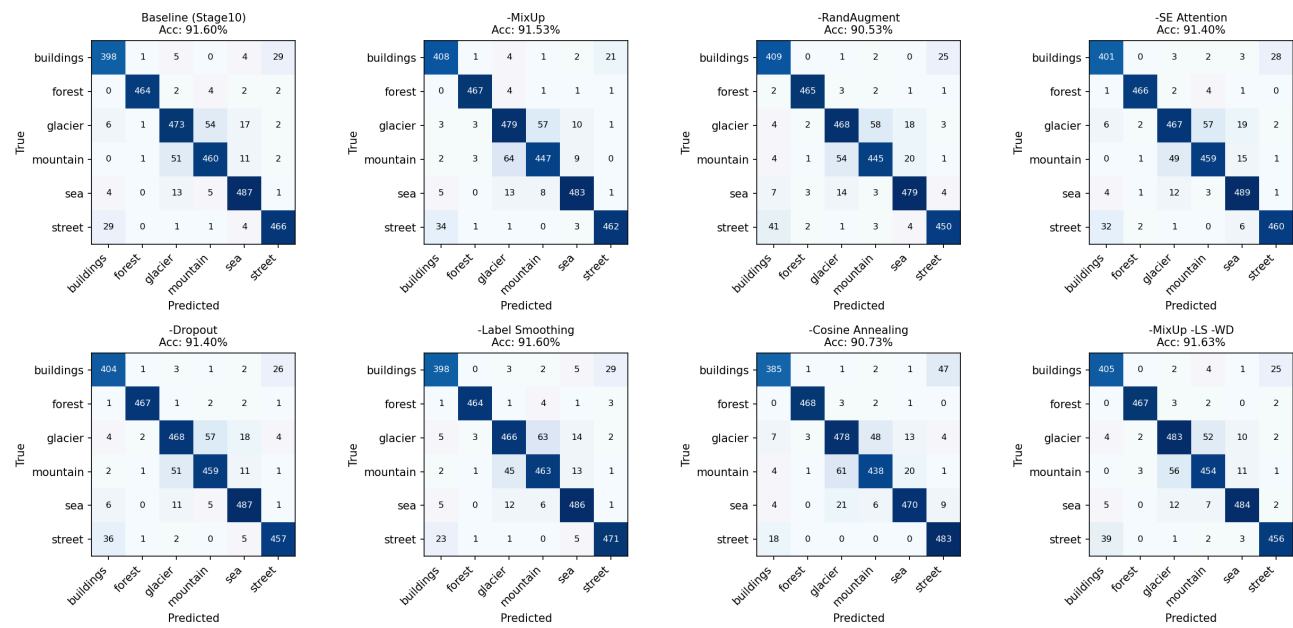
46 在测试集上评估...

47 测试准确率: 0.9163

48 实验完成,结果已保存。

消融实验总结 summarize_ablation

Model	Accuracy	Precision	Recall	F1-score
Baseline (Stage10)	0.916	0.9871092522137463	0.987	0.987000909693966
No MixUp	0.9153333333333333	0.9699010979129835	0.9696666666666667	0.9696651299520307
No RandAugment	0.9053333333333333	0.9679208833765021	0.9673333333333334	0.9673561300848558
No SE Attention	0.914	0.984027223076417	0.9836666666666667	0.9836872936199038
No Dropout	0.914	0.984309548538879	0.984	0.9840079089685235
No Label Smoothing	0.916	0.9769973280034986	0.9766666666666667	0.9766698428735009
No Cosine Annealing	0.9073333333333333	0.9579487838488117	0.957	0.956976263163328
No MixUp-LS-WD	0.9163333333333333	0.9751673458043578	0.975	0.9750032972731674



模型	准确率	变化	关键发现
Baseline (Stage10)	91.60%	—	基准
-MixUp	91.53%	-0.07%	影响极小，几乎可忽略
-RandAugment	90.53%	-1.07%	最大影响 ，数据增强的核心
-SE Attention	91.40%	-0.20%	影响较小，但正向
-Dropout	91.40%	-0.20%	影响较小，但正向
-Label Smoothing	91.60%	0.00%	无影响，可移除
-Cosine Annealing	90.73%	-0.87%	第二大影响 ，学习率调度关键
-MixUp -LS -WD	91.63%	+0.03%	略高于基准 ，说明这三者可安全移除

1. 最重要的两个组件：

- **RandAugment** (-1.07%)：移除后准确率下降最显著，证明强大的数据增强是性能提升的关键。
- **余弦退火** (-0.87%)：学习率调度同样重要，固定学习率会导致模型难以收敛到最优。

2. 贡献较小但仍正向的组件：

- SE 注意力、Dropout 分别贡献约 0.2%，说明它们提供了额外的正则化和特征重标定能力，但非必需。
- MixUp 仅贡献 0.07%，在当前强增强组合下已显冗余。

3. 可安全移除的组件：

- 标签平滑 (0.00% 影响) 在本配置下无增益，可去掉。
- 权重衰减 (未单独实验，但组合消融显示与 MixUp、标签平滑同时去掉后性能未降) 可能也属冗余。

4. 组合消融结果 (-MixUp -LS -WD) 准确率 91.63%，甚至略高于基准，说明：

- 这三个组件在当前配置下可能相互干扰或贡献微弱，移除后模型更简洁且性能稳定。
- 也可能由于随机波动 ($\pm 0.05\%$)，实际与基准持平。